

**UNIVERSIDADE CATÓLICA DE PETRÓPOLIS
CENTRO DE ENGENHARIA E COMPUTAÇÃO
CURSO DE ENGENHARIA DA COMPUTAÇÃO**

**Redes neurais aplicadas na predição de casos de
diabetes**

João Antonio Barcelos Coutinho Neto

Petrópolis
10 de Dezembro de 2025

Redes neurais aplicadas na predição de casos de diabetes

Trabalho de conclusão de curso, apresentado ao Centro de Engenharia e Computação da Universidade Católica de Petrópolis como requisito parcial para conclusão do Curso de Engenharia da Computação.

João Antonio Barcelos Coutinho Neto

Prof. Orientador
Fábio Lopes Licht

Petrópolis
10 de Dezembro de 2025

Agradecimentos

Dedico minha imensa gratidão a todos que, de alguma forma, participaram e foram cruciais para o meu crescimento e para a concretização desta etapa. Um agradecimento especial à minha família, que representou o meu alicerce, fornecendo apoio incondicional, estímulo e compreensão durante toda a jornada acadêmica.

Expresso meu reconhecimento também aos professores que me transmitiram seus conhecimentos e, de forma especial, ao meu orientador, Professor Fábio Lopes Licht, por ter aceitado minha proposta e dedicado seu tempo para me guiar na elaboração deste trabalho.

Resumo

O diabetes é uma doença crônica grave e prevalente, especialmente o Tipo 2, associada ao estilo de vida e que representa um desafio crescente para a saúde pública global. Neste contexto, as redes neurais artificiais surgem como uma ferramenta promissora para a detecção e prevenção, analisando grandes volumes de dados clínicos e comportamentais para identificar padrões complexos e classificar com precisão o risco individual. A integração desses modelos em sistemas de apoio à decisão médica potencia diagnósticos mais rápidos e o desenvolvimento de estratégias de intervenção precoce, visando reduzir a incidência e o impacto da doença.

Palavras-chave: Redes neurais, diabetes, predição, inteligência artificial, aprendizado de máquina.

Abstract

Diabetes is a serious and prevalent chronic disease, particularly Type 2, which is often linked to lifestyle factors and represents a growing challenge for global public health. In this context, artificial neural networks are emerging as a promising tool for detection and prevention, capable of analyzing large volumes of clinical and behavioral data to identify complex patterns and accurately classify an individual's risk. Integrating these models into clinical decision support systems enhances the potential for faster diagnoses and the development of early intervention strategies, aiming to reduce the disease's incidence and impact.

Keywords: Neural networks, diabetes, prediction, artificial intelligence, machine learning.

Lista de ilustrações

Figura 1 – Diferença entre classificação e regressão	22
Figura 2 – Ilustração de uma rede neural	25
Figura 3 – Gráfico da Função de Ativação Sigmóide.	27
Figura 4 – Gráfico da Função de Ativação Tangente Hiperbólica (Tanh).	28
Figura 5 – Gráfico da Função de Ativação ReLU.	28
Figura 6 – Interface principal do PyCharm	30
Figura 7 – Exemplo da integração do PyCharm com Jupyter Notebooks.	31
Figura 8 – Exemplo de gráfico gerado com Matplotlib.	35
Figura 9 – Distribuição da variável alvo diabetes no Diabetes Prediction Dataset.	44
Figura 10 – Distribuição ajustada com SMOTE.	45
Figura 11 – Matriz de confusão.	46
Figura 12 – Matriz de Confusão.	53
Figura 13 – Resultados: Curva Precision Recall.	54

Lista de tabelas

Tabela 1 – Descrição das Características usadas no conjunto de dados.	43
Tabela 2 – Resultados Trials MPL	55
Tabela 3 – Resultados Trials CNN	55
Tabela 4 – Resultados do treinamento CNN	56
Tabela 5 – Resultados do treinamento MLP	56
Tabela 6 – Resultados do tuning do modelo CNN	65
Tabela 7 – Resultados do tuning do modelo MLP	67

Lista de abreviaturas e siglas

AI	Artificial Intelligence (Inteligência Artificial)
ANN	Artificial Neural Network (Rede Neural Artificial)
AUC	Area Under the Curve
CSV	Comma-Separated Values
DM	Diabetes Mellitus
FN	Falso Negativo
FP	Falso Positivo
ML	Machine Learning (Aprendizado de Máquina)
PCA	Principal Component Analysis (Análise de Componentes Principais)
PRC	Precision-Recall Curve
RN	Rede Neural
ROC	Receiver Operating Characteristic
TN	Verdadeiro Negativo
TP	Verdadeiro Positivo
BMI	Body Mass Index (Índice de Massa Corporal)
HbA1c	Hemoglobina Glicada
IMC	Índice de Massa Corporal
IDE	Integrated Development Environment

Sumário

1	INTRODUÇÃO	17
1.1	Problematização	17
1.2	Solução Proposta	18
1.2.1	Delimitação do Escopo	18
1.2.2	Justificativa	18
1.3	Objetivos	19
1.3.1	Objetivo Geral	19
1.3.2	Objetivos Específicos	19
2	REFERENCIAL TEÓRICO	21
2.1	Diabetes	21
2.2	Machine Learning	21
2.2.1	Aprendizado Supervisionado	21
2.2.2	Aprendizado Não Supervisionado	22
2.2.3	Aprendizado Semi-Supervisionado	22
2.2.4	Aprendizado por Reforço	22
2.3	Machine Learning aplicado à Saúde	23
2.3.1	Classificação e Diagnóstico de Doenças	23
2.3.2	Suporte à Decisão Clínica	23
2.3.3	Medicina Personalizada	23
2.3.4	Monitoramento de Pacientes e Saúde Preventiva	24
2.4	Redes Neurais Artificiais	24
2.4.1	Funções De Ativação	26
2.5	IDE: PyCharm	29
2.6	Python e Bibliotecas para Machine Learning	32
2.6.1	Pandas	32
2.6.2	NumPy	34
2.6.3	Matplotlib e Seaborn	34
2.6.4	Scikit Learn	36
2.7	Tensor Flow e Keras	37
2.8	Limitações e Ética	38
3	TRABALHOS CORRELATOS	39
3.1	Uma abordagem baseada em redes neurais artificiais para diagnóstico de diabetes	39

3.2	Tecnologia aplicada à saúde: uso de inteligência artificial na predição de diabetes	39
3.3	Sistema Inteligente para Apoio ao Diagnóstico de Diabetes Em- pregando Redes Neurais	40
3.4	Diabetes Prediction: A Deep Learning Approach	40
4	DESENVOLVIMENTO	43
4.1	Dados	43
4.1.1	Normalização dos Dados	44
4.1.2	Divisão dos Dados De Treinamento e Teste	45
4.2	Métricas De Detecção de Qualidade	45
4.2.1	Matriz De Confusão	46
4.2.2	Acurácia	47
4.2.3	Loss	48
4.2.4	Precisão	48
4.2.5	Recall	49
4.2.6	F1 Score	49
4.3	Construção Dos Modelos	50
4.3.1	Otimização Bayesiana Aplicada Seleção de Hiperparâmetros	50
4.3.2	Avaliação dos modelos	52
4.4	Resultados preliminares e Discussão	54
4.4.1	Desempenho	54
4.4.2	Treinamento	56
5	CONCLUSÃO	59
	REFERÊNCIAS	61
	APÊNDICE A – TRIALS CNN	65
	APÊNDICE B – TRIALS MLP	67

1 Introdução

O diabetes mellitus é uma das doenças crônicas mais prevalentes no mundo, afetando milhões de pessoas e representando uma preocupação crescente para os sistemas de saúde pública (SBD, 2025; WHO, 2016; IDF, 2021). Seu diagnóstico precoce e tratamento adequado são essenciais para evitar complicações graves, como doenças cardiovasculares, falência renal, retinopatia e neuropatia. Dentre os tipos existentes, o diabetes tipo 2 é o mais comum, estando frequentemente associado a fatores comportamentais e ambientais, como má alimentação, sedentarismo e obesidade (IDF, 2021).

Com o avanço da tecnologia e o crescimento exponencial da disponibilidade de dados médicos e comportamentais, surgem novas oportunidades para apoiar o diagnóstico e a prevenção de doenças por meio da inteligência artificial. Entre as abordagens mais promissoras nesse campo estão as redes neurais artificiais, que integram o campo do aprendizado de máquina e se destacam por sua capacidade de identificar padrões complexos em grandes conjuntos de dados (MIOTTO et al., 2018; DEO, 2015).

Esses modelos podem analisar variáveis como idade, índice de massa corporal (IMC), níveis de glicose, histórico familiar e hábitos de vida, permitindo a criação de sistemas capazes de estimar o risco de um indivíduo desenvolver diabetes com alta precisão (SISODIA; SISODIA, 2018; KAVAKIOTIS et al., 2017). Essa capacidade torna as redes neurais uma ferramenta valiosa no apoio à decisão clínica, possibilitando diagnósticos mais rápidos, assertivos e estratégias de intervenção precoce.

É neste contexto que se insere o presente trabalho, ao propor a aplicação de redes neurais artificiais como uma solução tecnológica para a predição de casos de diabetes, contribuindo para a detecção precoce e melhoria na qualidade de vida dos pacientes.

1.1 Problematização

O diabetes tipo 2 representa atualmente um dos maiores desafios de saúde pública em todo o mundo, devido ao seu crescimento acelerado, alto potencial de morbidade e aos custos associados ao tratamento de complicações crônicas. (SBD, 2025; WHO, 2016; IDF, 2021). Apesar do conhecimento dos fatores de risco e dos avanços nos métodos de detecção, ainda há dificuldade significativa na identificação precoce de indivíduos propensos ao desenvolvimento da doença, especialmente em

contextos de grande demanda e recursos limitados. (IDF, 2021) Essa limitação reforça a necessidade de buscar soluções inovadoras que auxiliem na triagem, no diagnóstico precoce e na prevenção do diabetes tipo 2, permitindo melhorar a qualidade de vida dos pacientes e otimizar os recursos do sistema de saúde. (SISODIA; SISODIA, 2018; KAVAKIOTIS et al., 2017)

Nesse cenário, a integração de tecnologias baseadas em inteligência artificial se apresenta como uma alternativa promissora para superar desafios atuais e apoiar profissionais da saúde na tomada de decisão clínica.

1.2 Solução Proposta

Diante dessa realidade, este trabalho propõe o desenvolvimento e a avaliação de um modelo preditivo baseado em redes neurais artificiais. A intenção é utilizar dados clínicos e comportamentais acessíveis para produzir uma solução computacional capaz de auxiliar profissionais de saúde na identificação precoce de indivíduos sob risco de desenvolver diabetes tipo 2. O trabalho visa comparar modelos que utilizam técnicas modernas de aprendizado de máquina com algoritmos clássicos, buscando aumentar a precisão, agilidade e acessibilidade do apoio ao diagnóstico.

1.2.1 Delimitação do Escopo

Este estudo foca exclusivamente na aplicação de redes neurais para a predição de diabetes tipo 2 em adultos, utilizando um conjunto de dados secundários públicos, com variáveis essencialmente clínicas e comportamentais. O trabalho não se propõe a substituir o diagnóstico médico, mas sim oferecer uma ferramenta complementar para triagem e apoio à decisão clínica, respeitando os limites éticos e técnicos do uso da inteligência artificial na saúde.

1.2.2 Justificativa

A relevância do tema se justifica tanto pelo impacto epidemiológico do diabetes quanto pelo potencial das tecnologias de inteligência artificial em aprimorar processos de prevenção e diagnóstico em saúde pública. O desenvolvimento de modelos acessíveis e escaláveis pode contribuir para otimizar recursos, ampliar o alcance da triagem preventiva e melhorar o prognóstico dos pacientes devido à intervenção precoce. O estudo também proporciona aprendizado aplicado, integrando ciência de dados, programação e fundamentos de saúde.

1.3 Objetivos

Este trabalho visa desenvolver e avaliar um modelo de rede neural artificial capaz de prever casos de diabetes tipo 2 a partir da análise de dados clínicos e comportamentais, com o intuito de apoiar o diagnóstico precoce e contribuir para estratégias de prevenção.

1.3.1 Objetivo Geral

Desenvolver e avaliar um modelo de rede neural artificial capaz de prever casos de diabetes tipo 2 com base em dados clínicos e comportamentais, de modo a apoiar a triagem e o diagnóstico precoce em ambientes de saúde.

1.3.2 Objetivos Específicos

Este trabalho possui como objetivos específicos:

- Realizar levantamento e análise do estado da arte sobre o uso de redes neurais e modelos de machine learning para predição de doenças crônicas.
- Pré-processar e analisar o conjunto de dados utilizado, identificando os atributos mais relevantes ao risco do diabetes tipo 2.
- Implementar e treinar modelos de rede neural artificial, ajustando parâmetros e avaliando desempenho.
- Comparar a precisão do modelo proposto com métricas clássicas e métodos alternativos, discutindo limitações e potencial de aplicação.
- Sugerir recomendações para aprimoramento da ferramenta e sua eventual integração em fluxos clínicos reais.

2 Referencial Teórico

2.1 Diabetes

O diabetes mellitus é uma doença crônica caracterizada pela elevação persistente dos níveis de glicose no sangue, resultante de defeitos na produção e/ou ação da insulina. Os tipos mais comuns são o diabetes tipo 1, causado pela destruição autoimune das células beta pancreáticas; o tipo 2, associado à resistência à insulina e mais comum em adultos; e o diabetes gestacional, que ocorre durante a gravidez ([BRASIL, 2022](#)). Segundo a Organização Mundial da Saúde ([WORLD HEALTH ORGANIZATION \(WHO\), 2021](#)), o diabetes afeta centenas de milhões de pessoas em todo o mundo, sendo responsável por elevado ônus social e econômico devido às complicações crônicas, como doenças cardiovasculares, nefropatias, neuropatias e retinopatias. A detecção precoce é importante, já que intervenções no início do curso clínico da doença podem retardar ou prevenir complicações graves. Por isso, métodos preditivos ganham destaque como ferramentas de apoio à saúde pública.

2.2 Machine Learning

Segundo Andriy Burkov([BURKOV, 2020](#)) em Machine Learning Engineering o aprendizado de Máquina (ML), é um subcampo dinâmico da inteligência artificial (IA), dedica-se ao desenvolvimento de sistemas que adquirem a capacidade de aprender com os dados. Em vez de depender de regras programadas explicitamente para cada cenário, esses sistemas são projetados para identificar padrões complexos, construir modelos preditivos e tomar decisões autônomas. Essa capacidade de "aprender" a partir da experiência, de forma semelhante ao processo cognitivo humano, é o que distingue o ML de abordagens computacionais mais tradicionais.

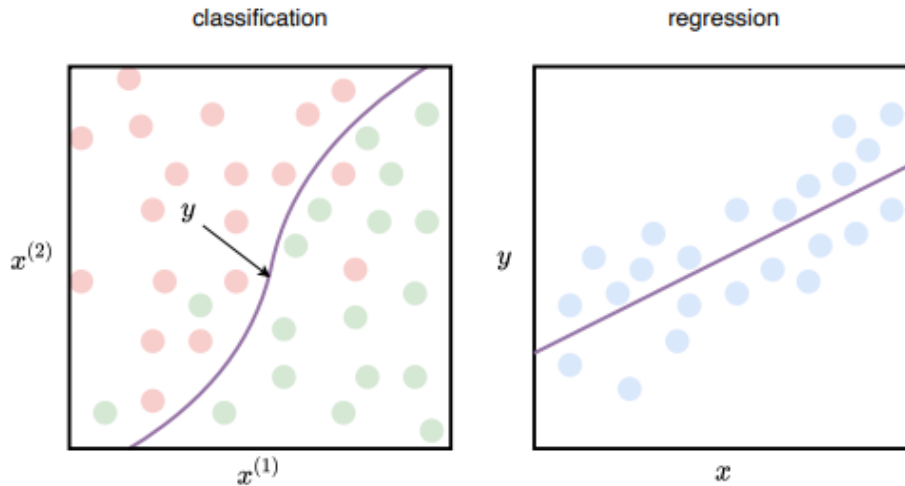
Tradicionalmente, as abordagens de ML são categorizadas em quatro paradigmas principais:

2.2.1 Aprendizado Supervisionado

Segundo Prado ([PRADO, 2001](#)), no aprendizado supervisionado, os algoritmos são treinados com conjuntos de dados rotulados, ou seja, onde as entradas já possuem as saídas corretas associadas. O objetivo é que o algoritmo aprenda a mapear essas entradas para as saídas, generalizando esse conhecimento para novos dados não vistos. Exemplos incluem classificação, como podem ver na figura 1 temos o exemplo

de predição de categorias discretas, como "doente" ou "saudável" e regressão predição de valores contínuos, como a pressão arterial. (FEITOSA; SOUZA; SOUZA, 2024)

Figura 1 – Diferença entre classificação e regressão



Fonte: (Burkov, 2020)

2.2.2 Aprendizado Não Supervisionado

Diferentemente do aprendizado supervisionado, no aprendizado não supervisionado, segundo Hastie e Friedman (HASTIE; TIBSHIRANI; FRIEDMAN, 2009), os dados utilizados nesta abordagem não possuem rótulos. O foco é descobrir estruturas ocultas, agrupamentos naturais ou padrões intrínsecos nos dados. Técnicas comuns incluem agrupamento, onde dados semelhantes são agrupados, e redução de dimensionalidade, que simplifica a representação dos dados mantendo suas características essenciais.

2.2.3 Aprendizado Semi-Supervisionado

Neste paradigma para Chapelle (CHAPELLE; SCHOLKOPF; ZIEN, 2006) ele representa um meio-termo entre o aprendizado supervisionado e o não supervisionado. Ele se torna particularmente útil em cenários onde a rotulagem de dados é um processo caro e demorado. Utiliza uma pequena quantidade de dados rotulados em conjunto com uma grande quantidade de dados não rotulados para treinar o modelo. A ideia é que os dados não rotulados ajudem o algoritmo a aprender sobre a estrutura geral dos dados, enquanto os dados rotulados fornecem a âncora para a aprendizagem direcionada.

2.2.4 Aprendizado por Reforço

Inspirado na psicologia comportamental, o aprendizado por reforço envolve um agente que interage com um ambiente dinâmico para alcançar um objetivo específico.

O agente aprende através de tentativas e erros, recebendo recompensas por ações desejáveis e penalidades por ações indesejáveis. O objetivo é que o agente aprenda uma política de ações que maximize a recompensa cumulativa ao longo do tempo. Esta abordagem é amplamente utilizada em robótica, jogos e sistemas de controle.

Conforme Mitchell ([MITCHELL, 1997](#)), a premissa fundamental do ML é permitir que os sistemas aprendam sem serem explicitamente programados para cada tarefa específica. Esta capacidade de adaptação e inferência a partir de dados é o que torna o ML uma ferramenta tão poderosa e versátil em uma gama crescente de domínios.

2.3 Machine Learning aplicado à Saúde

No setor da saúde, o Machine Learning tem demonstrado um potencial transformador, revolucionando diversas áreas e prometendo melhorias significativas na eficiência e qualidade dos cuidados. De acordo com ([ALZUBAIDI et al., 2021](#)), algumas das aplicações mais proeminentes incluem:

2.3.1 Classificação e Diagnóstico de Doenças

Algoritmos de ML podem analisar grandes volumes de dados de pacientes como por exemplo sintomas, resultados de exames laboratoriais e histórico médico para poderem auxiliar no diagnóstico precoce e preciso de doenças, desde condições crônicas como diabetes e doenças cardíacas até tipos específicos de câncer isso pode auxiliar a intervenções mais rápidas e eficazes.

2.3.2 Suporte à Decisão Clínica

Sistemas baseados em ML podem fornecer aos médicos informações valiosas e insights acionáveis para auxiliar na tomada de decisões clínicas. Isso inclui a recomendação de tratamentos personalizados com base nas características individuais do paciente, a previsão de resultados de tratamentos e a identificação de pacientes em risco de complicações.

2.3.3 Medicina Personalizada

Ao analisar o perfil genético, o histórico de vida e os dados clínicos de um indivíduo, o ML pode possibilitar abordagens de tratamento personalizadas, otimizando a escolha de terapias e dosagens para maximizar a eficácia e minimizar os efeitos colaterais.

2.3.4 Monitoramento de Pacientes e Saúde Preventiva

Dispositivos vestíveis e sensores podem coletar dados contínuos sobre a saúde dos indivíduos. Algoritmos de ML podem analisar esses dados para monitorar o estado de saúde, prever a ocorrência de eventos adversos e fornecer alertas em tempo real, permitindo uma intervenção precoce e promovendo a saúde preventiva.

O Aprendizado de Máquina está se consolidando como uma ferramenta indispensável no avanço da saúde, prometendo um futuro onde o diagnóstico é mais rápido, o tratamento mais eficaz e a medicina mais personalizada e preventiva.

2.4 Redes Neurais Artificiais

Uma Rede Neural Artificial (RNA) é um sistema computacional concebido para emular o modo como o cérebro humano processa informações ao executar uma tarefa específica. A sua implementação pode ser física, com o uso de componentes eletrônicos, ou, mais frequentemente, realizada através de simulação em computadores digitais. Para atingir um desempenho robusto, esses sistemas dependem de uma vasta rede interconectada de unidades computacionais elementares, conhecidas como "neurônios" ou unidades de processamento (HAYKIN, 2009).

Através de um processamento distribuído entre seus neurônios, as RNAs conseguem solucionar problemas de alta complexidade, mesmo em cenários onde o comportamento das variáveis subjacentes não é totalmente conhecido (BUSNARDO, 2024). Uma das suas características centrais é a habilidade de aprendizado por meio de exemplos, seguida da capacidade de generalizar o conhecimento adquirido para novos dados. Isso resulta na criação de um modelo não-linear, o que torna sua aplicação em domínios como a análise espacial altamente eficiente (SPÖRL; CASTRO; LUCHIARI, 2011).

No que tange à topologia, a implementação de uma RNA requer a definição de múltiplos parâmetros estruturais, entre os quais para santos(SANTOS, 2005) se destacam:

- O número de nós na camada de entrada, que corresponde à dimensionalidade dos dados que alimentam a rede, geralmente selecionadas pela sua importância para o problema
- A quantidade de camadas ocultas e o número de neurônios em cada uma delas.
- O número de neurônios na camada de saída, que é ditado pelo formato esperado da resposta do modelo.

As RNAs são, essencialmente, algoritmos computacionais que utilizam um modelo matemático inspirado na estrutura de organismos inteligentes, permitindo uma representação simplificada do funcionamento cerebral em um computador. Similarmente ao cérebro, a RNA é capaz de aprender e de tomar decisões fundamentadas nesse aprendizado. Trata-se, portanto, de um esquema de processamento que armazena conhecimento adquirido por treinamento e o disponibiliza para a aplicação a que se destina (SPÖRL; CASTRO; LUCHIARI, 2011).

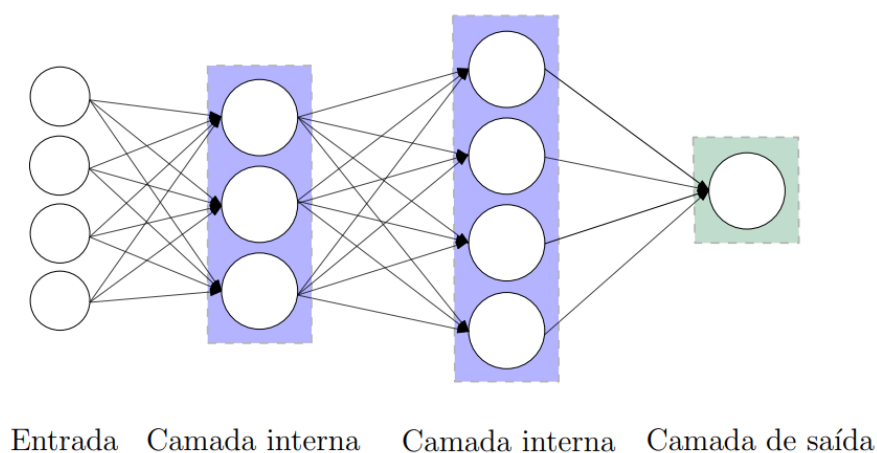
Conforme (HAYKIN, 2009), a analogia entre a rede neural e o cérebro humano se manifesta em dois aspectos fundamentais:

- o conhecimento é assimilado pela rede a partir de seu ambiente através de um processo de aprendizagem.
- As forças de conexão entre os neurônios, denominadas pesos sinápticos, são usadas para armazenar o conhecimento adquirido.

As redes neurais artificiais emergiram da generalização de modelos matemáticos de sistemas nervosos biológicos. Nessa abstração, os nós são chamados de neurônios, e as sinapses são modeladas como pesos nas conexões, que modulam o sinal de entrada de cada neurônio. A característica não-linear, inspirada nos neurônios biológicos, é introduzida por uma função de ativação, permitindo à rede modelar relações complexas (ABRAHAM, 2005).

Na arquitetura básica de um neurônio artificial demonstrada na figura 2, podemos observar que ele opera sobre sinais de entrada, os quais são valores numéricos ponderados pelos pesos sinápticos, podendo ser oriundos de outros neurônios ou de uma fonte inicial de dados (BUSNARDO, 2024).

Figura 2 – Ilustração de uma rede neural



Fonte: (Kovaleski, 2018)

O funcionamento de um neurônio artificial, conforme ilustrado na figura, pode ser representado pela seguinte equação matemática:

$$y = \phi \left(\sum_{i=1}^n \omega_i x_i + b \right)$$

Nesta expressão, x_i representa os sinais de entrada, ω_i são os respectivos pesos sinápticos, b é um valor de polarização, ϕ é a função de ativação aplicada à soma ponderada, e y é o sinal de saída do neurônio (KOVALESKI et al., 2018).

Segundo Busnardo (BUSNARDO, 2024), os modelos que estruturam esses neurônios em camadas sequenciais são conhecidos como Multilayer Perceptron (MLP). Um MLP típico é composto por três tipos de camadas: a camada de entrada onde os sinais codificados são inseridos, as camadas ocultas onde os dados são processados e transformados e a camada de saída que gera o resultado final, como uma classificação.

Individualmente, um neurônio só é capaz de resolver problemas linearmente separáveis. A solução para problemas complexos exige a associação de múltiplos neurônios em várias camadas interconectadas. Essa abordagem de empilhamento de camadas é a base de arquiteturas mais sofisticadas, como as Redes Neurais Convolucionais (ABRAHAM, 2005), que são pilares no campo do deep learning (LECUN; BENGIO; HINTON, 2015).

2.4.1 Funções De Ativação

Diversas funções de ativação são empregadas, cada uma com suas características e adequações para diferentes cenários:

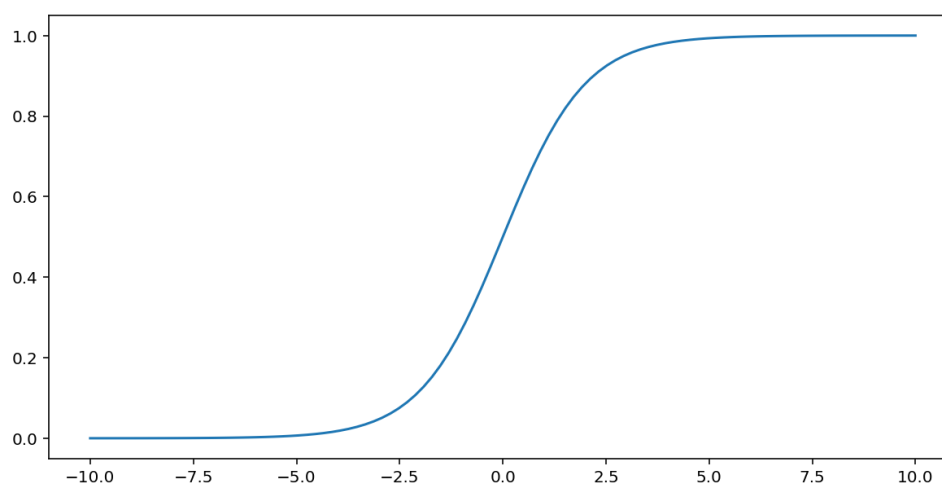
Função Sigmóide

Segundo (TRASK, 2019) a função sigmóide mapeia a entrada para um valor entre 0 e 1, sendo tradicionalmente usada em camadas de saída para problemas de classificação binária, onde a saída pode ser interpretada como uma probabilidade. No entanto, sofre do problema de "gradiente vanishing" para valores muito grandes ou muito pequenos, o que pode dificultar o treinamento de redes profundas. Sua função é definida da seguinte forma:

$$\phi(x) = \frac{1}{1 + e^{-x}}$$

Abaixo na figura 3 podemos observar o gráfico da função sigmoid.

Figura 3 – Gráfico da Função de Ativação Sigmóide.



Fonte: (GOOGLE, 2025))

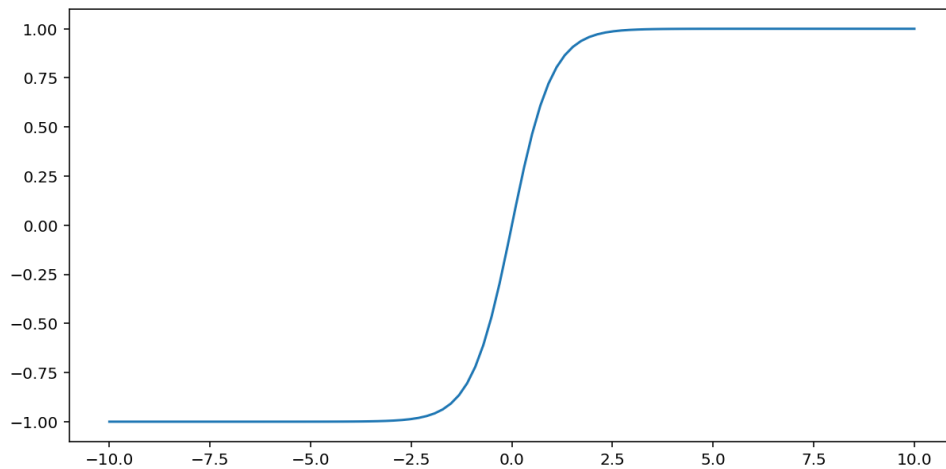
Função Tangente Hiperbólica (Tanh)

Tanh é uma função de ativação frequentemente utilizada em redes neurais, notadamente por sua capacidade de escalar as saídas para o intervalo $[-1, 1]$, o que contribui para a padronização dos valores de entrada. Sua característica de não-linearidade é fundamental, pois permite que a rede neural aborde problemas de classificação mais complexos, que não podem ser resolvidos por fronteiras lineares (GOODFELLOW; BENGIO; COURVILLE, 2016). De acordo com (GOODFELLOW; BENGIO; COURVILLE, 2016), a simetria do intervalo de saída em torno do zero é um fator que pode otimizar a velocidade de convergência do algoritmo de treinamento. Além disso, a derivabilidade da Tanh em todo o seu domínio simplifica significativamente o cálculo dos gradientes, um passo essencial no algoritmo de retropropagação. Por essas razões, a função de ativação Tanh é consistentemente uma escolha relevante em diversas aplicações de aprendizado de máquina com redes neurais. Sua função é definida da seguinte forma:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Abaixo na figura 4 podemos observar o gráfico da função Tanh.

Figura 4 – Gráfico da Função de Ativação Tangente Hiperbólica (Tanh).



Fonte: (GOOGLE, 2025))

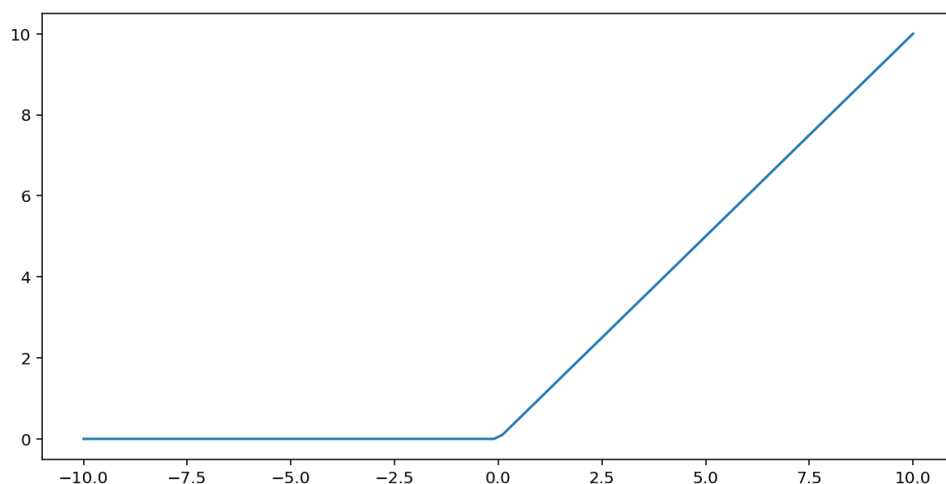
Função ReLU (Rectified Linear Unit)

Define a saída como o próprio valor de entrada se for positivo, e zero caso contrário. Essa função se tornou extremamente popular devido à sua simplicidade computacional e por mitigar o problema do gradiente vanishing para entradas positivas, acelerando o treinamento de redes profundas. No entanto, pode sofrer do problema "dying ReLU", onde neurônios podem se tornar inativos se suas entradas forem sempre negativas. A função ReLU é definida pela seguinte função:

$$f(x) = \max(0, x)$$

Abaixo na figura 5 podemos observar o gráfico da função ReLU.

Figura 5 – Gráfico da Função de Ativação ReLU.



Fonte: (GOOGLE, 2025))

Qual Função De Ativação Utilizar

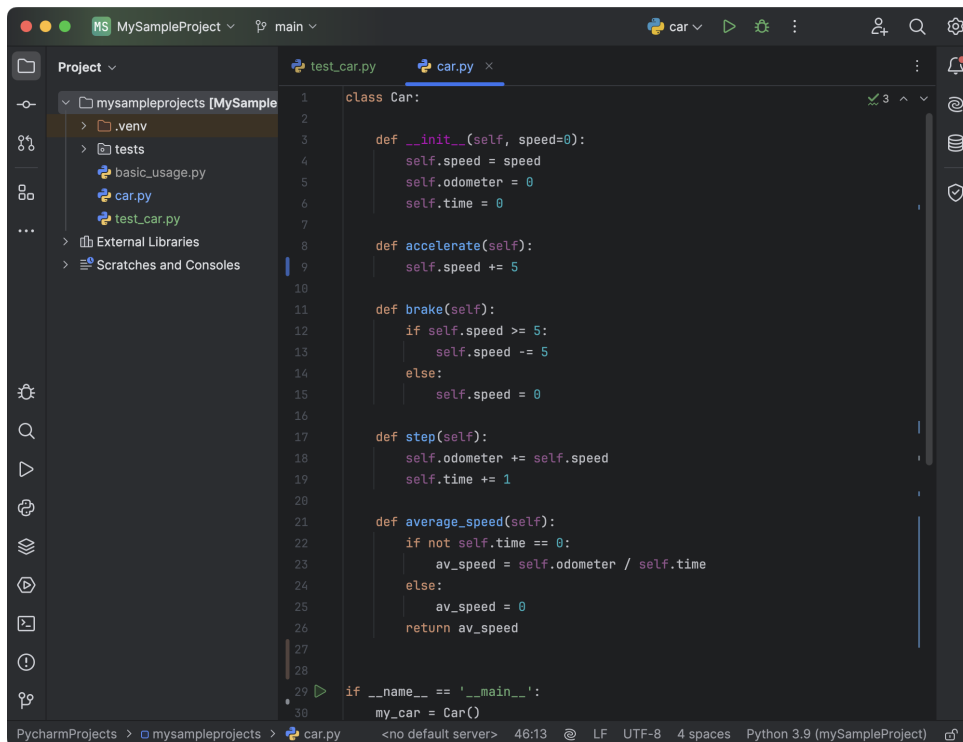
A escolha da função de ativação impacta diretamente a capacidade de aprendizado e o desempenho da rede neural. As funções sigmóide e tanh foram dominantes no passado, mas a ReLU e suas variantes como Leaky ReLU, ELU, PReLU são atualmente as mais utilizadas em camadas ocultas de redes profundas, devido à sua eficiência no treinamento.

A versatilidade das RNAs é evidenciada pela diversidade de modelos e arquiteturas que se desenvolveram a partir desse conceito fundamental. O Perceptron de Múltiplas Camadas (MLP), por exemplo, é uma das formas mais básicas e amplamente utilizadas, consistindo em camadas totalmente conectadas capazes de aprender mapeamentos complexos. Com o avanço da capacidade computacional e a disponibilidade de grandes volumes de dados, surgiram as redes neurais profundas (Deep Neural Networks), que se caracterizam por possuírem múltiplas camadas ocultas, permitindo a extração de representações de dados em diferentes níveis de abstração. As Redes Neurais Convolucionais (CNNs), por sua vez, destacam-se em problemas que envolvem dados com estrutura de grade, como imagens e sinais, utilizando filtros convolucionais para detectar padrões locais. Outros modelos especializados, como as Redes Neurais Recorrentes (RNNs) e suas variações LSTM, GRU, são particularmente eficazes no processamento de sequências temporais, como texto e áudio. Esses modelos vêm sendo amplamente utilizados em problemas que envolvem grandes volumes de dados e alta dimensionalidade, onde métodos tradicionais poderiam falhar em identificar padrões sutis.

2.5 IDE: PyCharm

O PyCharm é uma das IDEs (Integrated Development Environment) mais renomadas para a programação em Python, desenvolvida pela JetBrains. Ela se destaca pela combinação de uma interface intuitiva que observamos na figura 6 e uma ampla gama de recursos avançados, tornando-se uma ferramenta essencial para desenvolvedores de todos os níveis([??ROCHA, 2021](#); [PYLINTON, 2021](#)).

Figura 6 – Interface principal do PyCharm



Fonte: (JETBRAINS, 2025)

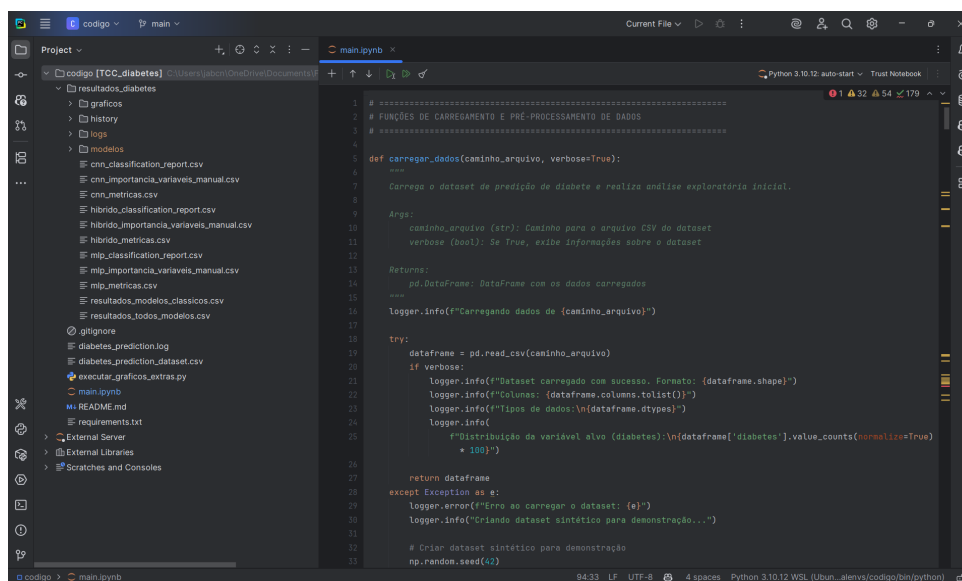
A seleção de um Ambiente de Desenvolvimento Integrado (IDE) é uma decisão fundamental que impacta a produtividade, a qualidade do código e a reprodutibilidade de um projeto de software. Para este trabalho, a escolha recaiu sobre o PyCharm, em detrimento de editores de código mais leves e genéricos, como o Visual Studio Code (VS Code). Essa decisão foi estratégica e baseada em uma análise das necessidades específicas de um projeto de machine learning de escopo acadêmico.

Enquanto o VS Code se destaca por sua leveza e versatilidade como um editor poliglota, sua funcionalidade para um ecossistema específico como o de Python depende da instalação e configuração manual de um conjunto de extensões para emular a experiência de um IDE.

O PyCharm, por outro lado, foi concebido especificamente para o desenvolvimento em Python, oferecendo uma experiência "out-of-the-box" coesa e otimizada, com todas as ferramentas essenciais pré-integradas e configuradas.

Em um projeto com prazo definido, como um Trabalho de Conclusão de Curso, o tempo economizado na configuração do ambiente de desenvolvimento pode ser reinvestido na lógica do problema, na experimentação e na análise de resultados.

Figura 7 – Exemplo da integração do PyCharm com Jupyter Notebooks.



Fonte: Elaborado pelo autor.

Três fatores foram preponderantes para esta escolha:

Depuração e Introspecção de Código: A complexidade de um pipeline de machine learning, que envolve múltiplas etapas de transformação de dados, treinamento de modelos e avaliação, torna a capacidade de depuração um requisito não-negociável. O depurador visual do PyCharm é nativamente superior para a ciência de dados. Ele permite a inspeção interativa de estruturas de dados complexas, como Data Frames do Pandas e arrays do NumPy, diretamente na interface gráfica durante a execução do código. A facilidade para definir breakpoints condicionais e executar o código passo a passo de forma intuitiva supera a experiência oferecida por configurações baseadas em extensões no VS Code, onde a depuração pode ser menos fluida.

Gerenciamento de Ambiente e Reprodutibilidade: A reprodutibilidade é um pilar da validade científica. O PyCharm oferece uma integração robusta para o gerenciamento de ambientes virtuais. A criação do ambiente, a ativação e a gestão de pacotes e suas versões são realizadas através de uma interface gráfica intuitiva, o que minimiza a probabilidade de erros de configuração. Esta abordagem garante que o ambiente de execução do projeto seja consistente, isolado e facilmente replicável por terceiros, um aspecto crucial para a validação dos resultados apresentados.

Integração com Ferramentas Científicas: A edição Profissional do PyCharm, utilizada neste projeto, oferece integração nativa com Jupyter Notebooks, conforme ilustrado na Figura 7. Isso possibilita um fluxo de trabalho híbrido e eficiente: a fase de Análise Exploratória de Dados pode ser conduzida de forma interativa em um notebook para rápida visualização e experimentação, enquanto o código de produção do pipeline é desenvolvido e depurado nos módulos .py padrão. Manter essas duas atividades

dentro de um único ambiente coeso elimina a fricção de alternar entre diferentes ferramentas.

Em suma, a escolha do PyCharm não foi uma questão de preferência pessoal, mas uma decisão de engenharia que visa mitigar riscos. Ao prover um ambiente coeso e pré-configurado, ele reduz a carga cognitiva do desenvolvedor, permitindo um foco maior na lógica do problema em vez de na configuração e manutenção das ferramentas de desenvolvimento. Esta abordagem eleva a qualidade e a robustez do trabalho, alinhando-o com boas práticas de engenharia de software.

2.6 Python e Bibliotecas para Machine Learning

Python é uma linguagem de programação de alto nível, de sintaxe simples e intuitiva, amplamente utilizada na ciência de dados e em projetos de inteligência artificial. Sua popularidade nessa área decorre, principalmente, do extenso ecossistema de bibliotecas especializadas, que simplificam as etapas do desenvolvimento de modelos de Machine Learning. A seguir, destacam-se algumas das principais tecnologias que compõem esse universo:

2.6.1 Pandas

Para a manipulação de dados estruturados, como o conjunto de dados de diabetes em formato CSV utilizado neste estudo, a biblioteca Pandas é o padrão da indústria e a escolha pragmática. Esta decisão foi tomada após a consideração de alternativas mais recentes e focadas em performance, como a biblioteca Polars.

O Pandas é uma biblioteca fundamental para manipulação e análise de dados. Ele fornece estruturas de dados eficientes, como Data Frames, que facilitam tarefas como limpeza, transformação, filtragem e agregação de dados, essenciais para o preparo das bases antes do treinamento dos modelos.(MCKINNEY, 2010)

Ainda segundo McKinney(MCKINNEY, 2010) a biblioteca oferece uma interface flexível para lidar com conjuntos de dados variados, disponibilizando recursos como agregação e visualização de dados, de forma intuitiva e eficiente.

A análise comparativa entre Pandas e Polars revela um clássico trade-off de engenharia de software. O Polars, implementado em Rust e projetado para paralelismo, oferece uma performance de execução significativamente superior, especialmente em operações de leitura/escrita e em datasets de grande volume. No entanto, o conjunto de dados deste projeto, com 100.000 instâncias, não é grande o suficiente para que a performance do Pandas se torne um gargalo crítico. O tempo de execução do pipeline de pré-processamento com Pandas permanece na ordem de segundos ou poucos

minutos, o que é perfeitamente aceitável para o projeto.

Neste contexto, a decisão de utilizar Pandas foi motivada pela otimização de um recurso mais escasso e valioso: o tempo de desenvolvimento. As vantagens do Pandas que justificaram sua escolha foram:

Maturidade e Integração do Ecossistema: A principal fortaleza do Pandas é sua integração nativa e transparente com as demais bibliotecas do projeto. Scikit-learn, Matplotlib e Seaborn foram desenvolvidos para interagir diretamente com as estruturas de dados do Pandas. Utilizar Polars exigiria conversões explícitas para o formato Pandas antes de passar os dados para essas bibliotecas, adicionando complexidade desnecessária, verbosidade ao código e um potencial ponto de falha no pipeline.

Produtividade e Suporte Comunitário: Sendo a ferramenta mais estabelecida, o Pandas possui uma vasta base de conhecimento, incluindo documentação oficial detalhada, inúmeros tutoriais e uma comunidade massiva que fornece soluções para praticamente qualquer problema de manipulação de dados. Essa riqueza de recursos acelera significativamente o desenvolvimento e a depuração, um fator crucial em projetos com cronogramas definidos.

O Pandas foi a espinha dorsal de toda a fase de exploração e pré-processamento de dados, com sua lógica centralizada no módulo de processamento de dados do projeto. Suas funcionalidades foram essenciais para:

- **Carregamento de Dados:** Ler o conjunto de dados original do arquivo CSV para um Data Frame.
- **Análise Exploratória de Dados (EDA):** Utilizar métodos nativos da biblioteca para obter uma compreensão inicial da estrutura, qualidade e distribuição dos dados.
- **Limpeza e Transformação:** Executar operações de limpeza, como a renomeação de colunas para o português e o tratamento de variáveis categóricas através de técnicas como
- **Engenharia de Features:** Criar, modificar ou remover colunas para preparar o conjunto de dados para os modelos de machine learning.
- **Segmentação dos Dados:** Separar o Data Frame principal em um conjunto de variáveis preditoras (X) e a variável alvo (y), que é o formato padrão para a maioria das APIs de machine learning.

Portanto, a escolha do Pandas reflete uma decisão que priorizou a velocidade de desenvolvimento, a robustez da integração e a minimização do atrito entre as

ferramentas em detrimento de uma otimização de performance de execução que traria benefícios marginais para a escala do problema em questão.

2.6.2 NumPy

A inclusão da biblioteca NumPy no projeto não representa uma escolha entre alternativas, mas sim a adoção de um pilar fundamental e indispensável. O NumPy é o alicerce sobre o qual todo o ecossistema de ciência de dados em Python é construído. Bibliotecas cruciais utilizadas neste trabalho, como Pandas, Scikit-learn e Tensor Flow, dependem intrinsecamente da estrutura de dados central do NumPy, o ndarray, para realizar operações numéricas de alta performance.

A eficiência do NumPy deriva de sua implementação em linguagens de baixo nível, que permite a execução de operações matemáticas em vetores e matrizes de forma muito mais rápida do que seria possível com laços em Python puro. Tentar substituir o NumPy significaria abandonar o vasto e maduro ecossistema de ferramentas científicas de Python ou reconstruir funcionalidades básicas de computação numérica, ambas opções inviáveis para o escopo deste trabalho.

Ao longo do projeto a biblioteca operou de forma transparente nos bastidores em quase todas as etapas. Quando um DataFrame do Pandas era criado, suas colunas numéricas eram armazenadas internamente como arrays NumPy. Da mesma forma, quando os dados eram passados para os modelos do Scikit-learn ou Keras para treinamento e predição, a estrutura de dados subjacente manipulada por esses frameworks era o ndarray do NumPy.

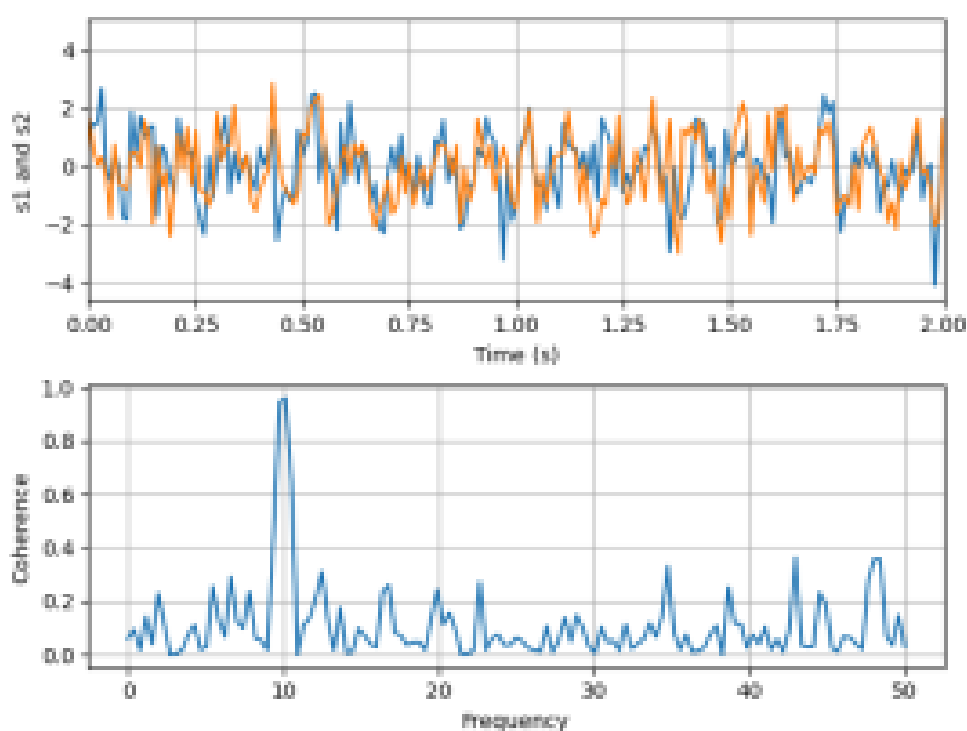
Em momentos específicos, a biblioteca foi invocada diretamente. No módulo de processamento de dados onde funções do NumPy foram utilizadas para realizar transformações matemáticas eficientes em colunas de dados. Além disso, na etapa final do pré-processamento, os dados preparados foram explicitamente convertidos para o formato de array NumPy, que é o formato de entrada padrão e otimizado exigido pela maioria dos algoritmos de machine learning, incluindo os implementados pelo Keras e Scikit-learn.

2.6.3 Matplotlib e Seaborn

A estratégia de visualização de dados para este trabalho foi deliberadamente focada na geração de gráficos estáticos de alta qualidade, utilizando a combinação sinérgica das bibliotecas Matplotlib e Seaborn. A escolha desta abordagem foi diretamente influenciada pelo meio de comunicação primário dos resultados do projeto: um documento de tese estático.

Foram consideradas alternativas como Plotly e Bokeh, bibliotecas que se destacam pela capacidade de criar visualizações interativas, ideais para dashboards e aplicações web. No entanto, a principal vantagem dessas ferramentas é a interatividade que é completamente perdida quando um gráfico é incorporado em um documento de texto ou PDF. A exportação de um gráfico interativo como uma imagem estática frequentemente resulta em uma qualidade visual inferior e menor controle sobre a formatação final em comparação com um gráfico gerado nativamente para esse propósito, temos um exemplo na figura 8.

Figura 8 – Exemplo de gráfico gerado com Matplotlib.



Fonte: matplotlib(2025))

O que justifica a escolha do Matplotlib e Seaborn pela complementaridade de suas funções:

Matplotlib como a Fundação Flexível: Matplotlib é a biblioteca de visualização mais fundamental e poderosa do ecossistema Python. Ela oferece um controle granular e de baixo nível sobre todos os elementos de uma figura como eixos, títulos, legendas, cores, fontes e anotações.

Seaborn para Produtividade e Estética Estatística: Seaborn funciona como uma API de alto nível construída sobre o Matplotlib. Sua principal função é simplificar a criação de gráficos estatísticos complexos e esteticamente agradáveis. Para a fase de Análise Exploratória de Dados, o Seaborn acelera drasticamente a obtenção de insights ao permitir a geração de visualizações ricas, como mapas de calor de correlação,

gráficos de distribuição e diagramas de dispersão, frequentemente com uma única linha de código.

A combinação das duas ferramentas oferece uma solução ótima juntando a velocidade e a simplicidade do Seaborn para a exploração inicial e a criação de gráficos padrão, e o poder de personalização do Matplotlib para os ajustes finos necessários para garantir que as figuras finais sejam informativas, precisas e visualmente claras. Essa abordagem demonstra um entendimento de que a visualização de dados não é apenas uma ferramenta de análise para o pesquisador, mas também um instrumento de comunicação para a audiência, onde a clareza e a adequação ao meio são primordiais.

2.6.4 Scikit Learn

A biblioteca Scikit learn é um dos principais frameworks para o desenvolvimento e a avaliação de modelos de Machine Learning tradicionais. Ela fornece algoritmos para classificação, regressão, clustering e redução de dimensionalidade, além de ferramentas para validação cruzada, seleção de atributos e ajuste de hiperparâmetros (PEDREGOSA et al., 2011).

A utilização da biblioteca Scikit-learn foi uma decisão metodológica fundamental para atender a um dos objetivos específicos do projeto: "Comparar a precisão do modelo proposto com métodos alternativos". A sua inclusão não foi opcional, mas sim um requisito para garantir o rigor científico da análise comparativa.

Scikit-learn é o framework padrão de fato para algoritmos de machine learning clássicos em Python. Sua escolha se baseia em três pilares:

Padrão da Indústria e API Consistente: A biblioteca oferece uma implementação robusta e otimizada de uma vasta gama de algoritmos de classificação, regressão e clusterização. Mais importante, ela o faz através de uma API consistente e unificada, que simplifica enormemente o processo de treinar e avaliar múltiplos modelos de forma comparável.

Estabelecimento de um Baseline Científico: Antes de justificar o uso de um modelo mais complexo e computacionalmente mais caro, como uma rede neural artificial, é uma prática essencial na pesquisa estabelecer um baseline de performance. Modelos mais simples e interpretáveis, como Regressão Logística ou Random Forest, fornecidos pelo Scikit-learn, servem como um ponto de referência crucial. Eles ajudam a quantificar a dificuldade intrínseca do problema e a determinar se a complexidade adicional de uma rede neural se traduz em um ganho de desempenho significativo.

Ferramentas de Avaliação Padronizadas: O módulo sklearn.metrics é considerado o padrão-ouro para a avaliação de modelos de machine learning. Ele fornece implementações eficientes e amplamente validadas de todas as métricas de desem-

penho críticas mencionadas no trabalho, como Acurácia, Precisão, Recall, F1-Score e a Matriz de Confusão. Utilizar este módulo para avaliar todos os modelos, incluindo a rede neural implementada com Keras, garante que a comparação seja justa, consistente e baseada em métricas calculadas de forma idêntica, eliminando vieses de implementação.

2.7 Tensor Flow e Keras

Para projetos que demandam aprendizado profundo, as bibliotecas Tensor Flow e Keras são amplamente utilizadas. O Tensor Flow, desenvolvido pela Google, oferece uma infraestrutura robusta para o treinamento de modelos complexos de redes neurais, enquanto o Keras, integrável ao Tensor Flow, fornece uma interface mais amigável, facilitando a definição e o treinamento de modelos de Deep Learning.

A decisão da escolha pela a API de alto nível Keras, utilizando o Tensor Flow como seu backend de computação foi tomada após uma análise comparativa com a principal alternativa no campo de deep learning, o PyTorch.

PyTorch é amplamente reconhecido por sua flexibilidade, sua API e seu grafo computacional dinâmico, características que o tornam a ferramenta preferida em ambientes de pesquisa acadêmica, onde a experimentação com novas e complexas arquiteturas de rede é comum. Keras, em contraste, foi desenvolvido com uma filosofia de simplicidade, modularidade e facilidade de uso, abstraindo grande parte da complexidade de baixo nível associada ao treinamento de redes neurais.

A escolha de Keras e Tensor Flow foi estratégica e alinhada ao escopo do projeto:

Foco na Aplicação, Não na Pesquisa de Arquiteturas: O objetivo deste TCC é a aplicação de uma arquitetura de rede neural conhecida como Perceptron de Múltiplas Camadas - MLP para resolver um problema prático, e não a pesquisa e o desenvolvimento de uma nova topologia de rede. Neste contexto, a abstração e a simplicidade do Keras são uma vantagem competitiva. Elas permitem que o desenvolvedor se concentre naquilo que é mais importante para o problema: a definição da arquitetura número de camadas, neurônios, funções de ativação e o ajuste de hiperparâmetros, em vez de se preocupar com a implementação manual de laços de treinamento, cálculo de gradientes e atualização de pesos, tarefas que são mais explícitas em PyTorch.

Produtividade e Curva de Aprendizagem: Em um projeto de graduação com um cronograma definido, a produtividade é um fator crítico. A curva de aprendizagem mais suave do Keras permite que um desenvolvedor se torne produtivo muito mais rapidamente. A capacidade de construir, compilar e treinar um modelo com poucas

linhas de código declarativo acelera o ciclo de experimentação, permitindo que mais tempo seja dedicado à análise dos resultados e ao refinamento do modelo.

A utilização conjunta de Scikit-learn e Keras constitui uma estratégia metodológica complementar e robusta. O Scikit-learn foi usado para estabelecer o piso de performance com modelos clássicos, definindo o baseline a ser superado. O Keras, por sua vez, foi empregado para testar a hipótese central do trabalho: que uma abordagem de deep learning, apesar de sua maior complexidade, poderia oferecer um desempenho preditivo superior para este problema específico. Essa dualidade de ferramentas transforma a seleção tecnológica em um desenho experimental estruturado, com um grupo de controle e um grupo de teste.

2.8 Limitações e Ética

A adoção de sistemas baseados em inteligência artificial na saúde exige atenção a diversas limitações e questões éticas. Devem ser considerados o respeito à privacidade e à confidencialidade dos dados dos pacientes, além das possíveis vieses presentes nos dados utilizados para o treinamento, que podem resultar em discriminações ou erros sistemáticos. Outro ponto relevante é a necessidade de transparência na interpretação dos resultados produzidos pelo modelo. A conformidade com normas éticas e regulatórias, bem como a análise criteriosa dos resultados preditivos por profissionais de saúde, são fundamentais para o uso responsável dessas tecnologias (BRASIL, 2022).

O trabalho desenvolvido tem como objetivo principal criar uma ferramenta de apoio ao profissional de saúde. Ressalta-se que essa ferramenta não substitui o profissional nem deve ser utilizada como diagnóstico final, mas sim como um auxílio adicional no processo de tomada de decisão clínica.

3 Trabalhos correlatos

Este capítulo apresenta quatro trabalhos relevantes para o tema da predição de diabetes utilizando inteligência artificial. Cada trabalho é descrito em detalhes, abordando o contexto, a solução proposta e as principais métricas obtidas. Ao final de cada seção, é feito um comparativo com o TCC em desenvolvimento, destacando diferenças e potenciais avanços.

3.1 Uma abordagem baseada em redes neurais artificiais para diagnóstico de diabetes

O trabalho de Araújo ([ARAÚJO, 2021](#)) propõe uma abordagem para diagnóstico de diabetes tipo 2 por meio de redes neurais artificiais. A base de dados utilizada foi a tradicional Pima Indians Diabetes Dataset, composta por informações clínicas de pacientes como glicose, IMC, idade, pressão arterial e número de gestações. O autor justifica a escolha da base por sua ampla aceitação acadêmica e foco em variáveis fisiológicas.

A técnica adotada foi o uso de uma rede neural MLP (Multilayer Perceptron), com o framework Tensor Flow. O autor aplicou técnicas de pré-processamento, como normalização dos dados e aplicação do algoritmo SMOTE para balanceamento. Os experimentos foram avaliados por meio de validação cruzada e apresentaram bons resultados, embora o trabalho não forneça uma tabela específica com métricas como acurácia ou F1-score.

Comparando com o presente TCC, destaca-se que este projeto utiliza um conjunto de dados mais abrangente, o Diabetes Prediction Dataset do Kaggle, que além das variáveis clínicas, inclui fatores comportamentais e de estilo de vida como frequência de exercícios, estresse, sono e alimentação. Isso amplia a capacidade preditiva do modelo. Além disso, são utilizadas arquiteturas de redes neurais modernas com potencial de melhor desempenho.

3.2 Tecnologia aplicada à saúde: uso de inteligência artificial na predição de diabetes

Santos ([SANTOS, 2024](#)) propôs o uso de algoritmos de aprendizado de máquina para prever diabetes tipo 2 em adultos. O foco do trabalho está na comparação de

desempenho entre diferentes classificadores: Random Forest, Regressão Logística, K-Nearest Neighbors e Árvore de Decisão. As ferramentas utilizadas foram R e Python, com ênfase em métricas como acurácia, sensibilidade e precisão.

A autora realizou uma análise sistemática dos modelos com diferentes divisões dos dados. Embora as métricas numéricas exatas não tenham sido especificadas com detalhes no resumo do trabalho, é indicado que os resultados foram satisfatórios e que o Random Forest apresentou desempenho superior entre os modelos testados.

Diferente do trabalho de Santos, este TCC propõe a utilização exclusiva de redes neurais artificiais para a predição, com foco em modelagem profunda (deep learning). Além disso, destaca-se o uso de dados mais recentes e diversificados, o que pode permitir maior generalização dos resultados.

3.3 Sistema Inteligente para Apoio ao Diagnóstico de Diabetes Empregando Redes Neurais

No projeto desenvolvido Basso ([BASSO et al., 2014](#)) apresenta um sistema inteligente para apoio ao diagnóstico de diabetes utilizando uma rede neural feedforward com múltiplas camadas. O sistema foi implementado em Java, com base de dados Pima Indians, e integração com o SQL Server para armazenamento dos dados. A arquitetura neural adotada possui uma camada oculta com 10 neurônios, utilizando o algoritmo de treinamento backpropagation.

Foram utilizados 538 pacientes para treinamento e 230 para teste. O sistema atingiu uma acurácia de 81,31%, com 187 acertos e 43 erros. O resultado foi considerado promissor, especialmente por superar trabalhos anteriores que utilizaram a mesma base. A normalização dos dados e a escolha empírica da arquitetura foram apontadas como pontos-chave do sucesso do modelo.

Este trabalho se diferencia por utilizar tecnologias mais atuais, além de dados com variáveis comportamentais, não presentes na base Pima. Espera-se, portanto, alcançar maior desempenho preditivo, além de facilitar a integração do modelo a ambientes modernos e sistemas clínicos reais.

3.4 Diabetes Prediction: A Deep Learning Approach

No artigo desenvolvido por Islam e Islam([ISLAM; ISLAM, 2019](#)), aplica-se uma abordagem de Deep Neural Network (DNN) para a predição de diabetes tipo 2, para dados tabulares. A base de dados utilizada também foi a Pima Indians, e a rede implementada possui quatro camadas ocultas com 12, 16, 16 e 14 neurônios respectivamente.

O modelo foi avaliado com validação cruzada 5-fold e 10-fold.

Os resultados obtidos foram bastante expressivos: 98,35% de acurácia, 97,39% de sensibilidade e 99,80% de especificidade com validação 5-fold. Na validação 10-fold, a acurácia foi de 97,27%. As ferramentas utilizadas incluíram Scikit-learn e a IDE Spyder, em Python.

Embora o desempenho seja notável, o presente trabalho propõe utilizar uma base com maior diversidade de variáveis, o que pode melhorar a aplicabilidade e a robustez do modelo preditivo, o trabalho atual também propõe fazer a comparação entre os modelos treinados utilizando algoritmos clássicos de machine learning e a comparação com modelos de redes neurais.

4 Desenvolvimento

Esta seção detalha o processo de desenvolvimento do modelo de predição de diabetes, abordando a arquitetura do projeto, o fluxo de execução do pipeline, a funcionalidade de cada módulo de código, as tecnologias e bibliotecas empregadas, e os resultados obtidos. O objetivo é fornecer uma compreensão aprofundada da implementação técnica e das decisões tomadas para construir uma solução robusta e eficaz na identificação precoce de casos de diabetes tipo 2.

4.1 Dados

O conjunto de dados selecionado para este estudo foi o Diabetes Prediction Dataset, disponibilizado na plataforma Kaggle por lammustafa(IAMMUSTAFA, 2022). O dataset é composto por registros de pacientes, com diversas variáveis clínicas e demográficas que visam prever a presença ou ausência de diabetes, ele é composto por 100.000 registros, cada um representando um indivíduo com diversas características clínicas e comportamentais, e uma variável alvo indicando a presença ou ausência de diabetes.

O conjunto de dados apresenta características clínicas que serão detalhadas na Tabela 1.

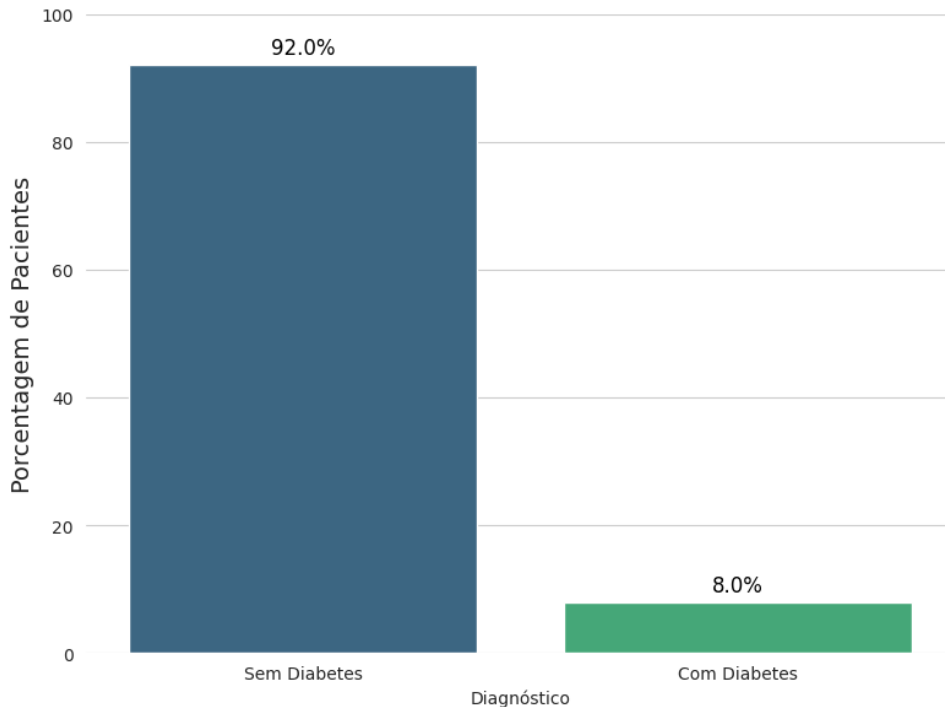
Tabela 1 – Descrição das Características usadas no conjunto de dados.

Característica	Descrição
gender	gênero
age	idade
hypertension	hipertensão
heart disease	doença cardíaca
smoking history	histórico de tabagismo
bmi	Índice de Massa Corporal
HbA1c level	nível de Hemoglobina Glicada
blood glucose level	nível de glicose no sangue
diabetes	"variável alvo: 0 para não diabético, 1 para diabético"

É relevante destacar que o dataset reflete uma distribuição de classes significativa, onde a maioria das instâncias corresponde a indivíduos sem diabetes. A variável alvo diabetes apresenta um desbalanceamento, com 8% dos pacientes diagnosticados com diabetes e 92% sem o diagnóstico essa distribuição é demonstrada na figura 10. Essa característica do dataset é crucial para o processo de modelagem, exigindo a

aplicação de técnicas específicas para mitigar o viés em favor da classe majoritária, como o SMOTE, que será detalhado na seção 4.1.1.

Figura 9 – Distribuição da variável alvo diabetes no Diabetes Prediction Dataset.



Fonte: Elaborado pelo autor

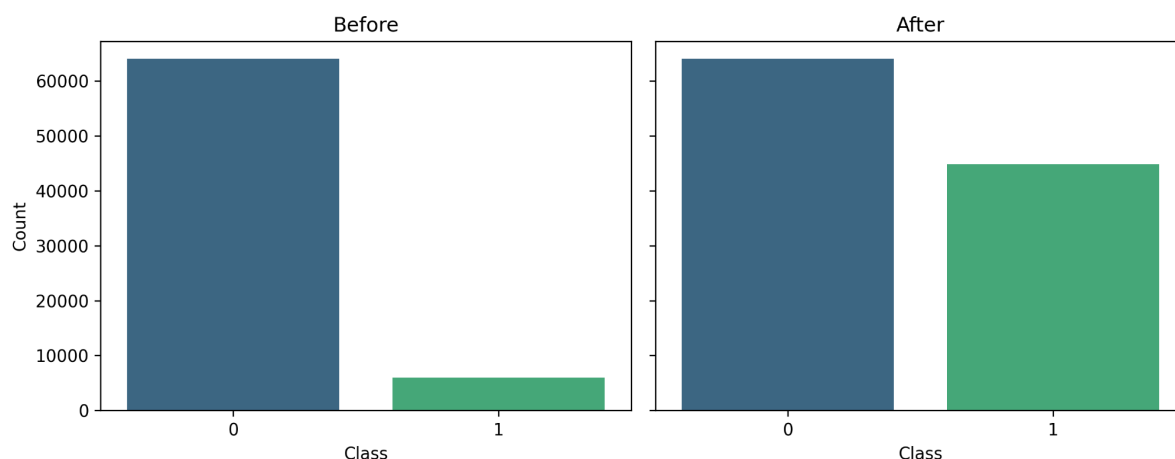
A qualidade e a abrangência das informações contidas neste dataset o tornam um recurso valioso para a aplicação de técnicas de aprendizado de máquina na predição de doenças, oferecendo um cenário realista para o desenvolvimento e validação de modelos preditivos em saúde.

4.1.1 Normalização dos Dados

A normalização dos dados foi uma etapa essencial do pré-processamento e utilizou a padronização por Z-score, que transforma cada atributo para média 0 e desvio padrão 1. No pipeline, essa padronização é aplicada após o tratamento de outliers numéricos via limites de IQR, garantindo que todas as features entrem no modelo na mesma escala.

A técnica SMOTE também foi aplicada para mitigar o desbalanceamento de classes no conjunto de treino. Ela cria amostras sintéticas da classe minoritária, após tratamento de outliers, codificação one-hot e padronização por Z-score, evitando vazamento de informação, os conjuntos de validação e teste não foram reajustados. Esta estratégia aumentou a representatividade da classe minoritária como podemos ver na figura 11, e reduziu o viés de previsão e melhorou métricas como recall e AUC-ROC, sem comprometer a avaliação do modelo.

Figura 10 – Distribuição ajustada com SMOTE.



Fonte: Elaborado pelo autor

A normalização de dados é fundamental para otimizar o desempenho do modelo, pois padroniza a escala dos recursos. Essa padronização melhora a interpretabilidade, acelera a convergência e previne problemas de gradiente causados por magnitudes desiguais. Além disso, a normalização favorece a comparabilidade e a interpretabilidade de coeficientes e importâncias de atributos, uma vez que coloca os atributos na mesma unidade, contribuindo para um desempenho mais robusto dos modelos.

4.1.2 Divisão dos Dados De Treinamento e Teste

Para o treinamento, o conjunto de dados foi dividido, dimensionado e transformado em subconjuntos de treinamento e teste, utilizando a função `train_test_split` da biblioteca `scikit-learn`.

Do conjunto de dados original, 20% foram reservados para testes. O restante foi então dividido de forma estratificada, com 70% para treino e 10% para validação. A utilização de uma semente aleatória fixa garantiu a reprodutibilidade dos resultados. Essa metodologia permite uma avaliação imparcial com dados nunca antes vistos e a calibração precisa dos modelos através do conjunto de validação.

4.2 Métricas De Detecção de Qualidade

A avaliação do desempenho de um modelo de Machine Learning é um passo crítico para garantir sua eficácia e confiabilidade, especialmente em tarefas de classificação como a predição de diabetes. As métricas de detecção de qualidade fornecem uma quantificação do quão bem o modelo está realizando suas previsões, permitindo a comparação entre diferentes abordagens e a identificação de áreas para melhoria. A escolha das métricas apropriadas depende do problema em questão e das característi-

cas do conjunto de dados, como o balanceamento das classes (HAN; KAMBER; PEI, 2012; BISHOP, 2006).

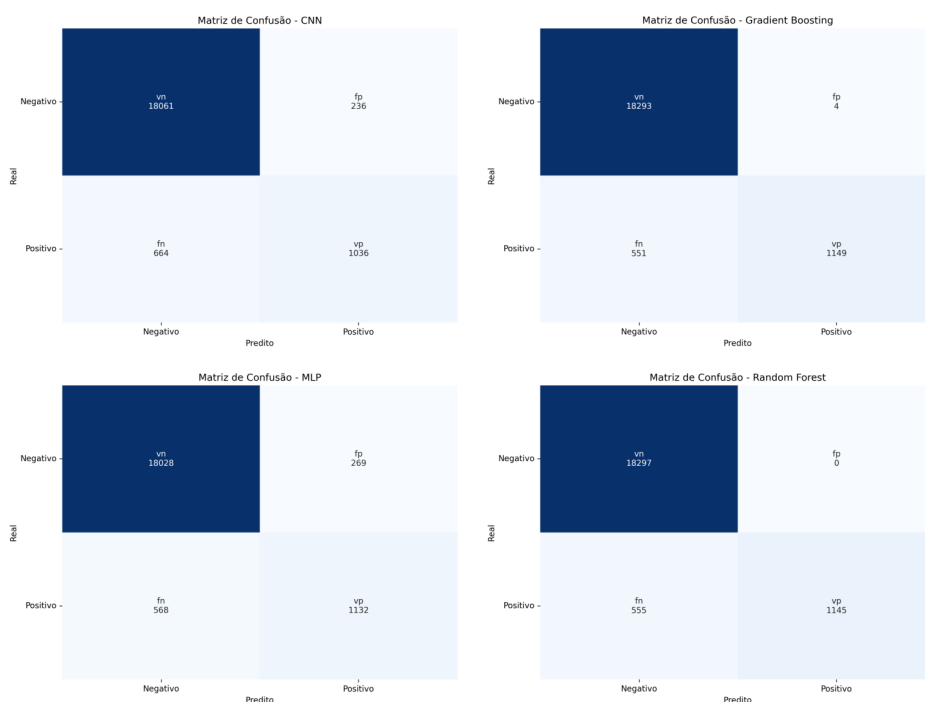
Para modelos de classificação, as métricas são frequentemente derivadas da Matriz de Confusão, que detalha os resultados das previsões em relação aos valores reais. As principais métricas incluem Acurácia, Precisão, Recall e F1-Score, cada uma oferecendo uma perspectiva diferente sobre o desempenho do modelo (FREITAS, 2024). Para problemas de regressão, como a predição de um valor contínuo, métricas como o RMSE são mais adequadas. A seguir, serão detalhadas as métricas relevantes para a avaliação do modelo de predição de diabetes.

4.2.1 Matriz De Confusão

A Matriz de Confusão é uma tabela que sumariza o desempenho de um algoritmo de classificação, apresentando a contagem de previsões corretas e incorretas feitas pelo modelo em comparação com os valores reais. Ela é particularmente útil para entender os tipos de erros que o modelo está cometendo, especialmente em problemas de classificação binária (STRAUSS; JÚNIOR; FERREIRA, 2022).

A matriz é composta por quatro elementos principais:

Figura 11 – Matriz de confusão.



Fonte: Elaborado pelo autor.)

- **Verdadeiro Positivo (VP):** O modelo previu corretamente a classe positiva (ex: previu que o indivíduo tem diabetes e ele realmente tem).

- **Verdadeiro Negativo (VN):** O modelo previu corretamente a classe negativa (ex: previu que o indivíduo não tem diabetes e ele realmente não tem).
- **Falso Positivo (FP):** O modelo previu incorretamente a classe positiva por exemplo previu que o indivíduo tem diabetes, mas ele não tem.
- **Falso Negativo (FN):** O modelo previu incorretamente a classe negativa como por exemplo, previu que o indivíduo não tem diabetes, mas ele tem.

A estrutura da Matriz de Confusão é a seguinte:

	Previsto Positivo	Previsto Negativo
Real Positivo	Verdadeiro Positivo (VP)	Falso Negativo (FN)
Real Negativo	Falso Positivo (FP)	Verdadeiro Negativo (VN)

Através da Matriz de Confusão, é possível calcular diversas métricas de desempenho que fornecem uma visão mais aprofundada sobre a qualidade do modelo, indo além da simples acurácia ([PIGHINI, 2025](#)).

4.2.2 Acurácia

A Acurácia é a métrica mais intuitiva e amplamente utilizada para avaliar o desempenho de um modelo de classificação. Ela representa a proporção de previsões corretas em relação ao total de previsões realizadas. Em outras palavras, a acurácia mede a porcentagem de vezes que o modelo fez a previsão correta. A fórmula para calcular a Acurácia é:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{VP + VN}{VP + VN + FP + FN}$$

Onde:

- VP = Verdadeiro Positivo
- VN = Verdadeiro Negativo
- FN = Falso Negativo
- FP = Falso Positivo

Embora a acurácia seja fácil de entender e calcular, ela pode ser enganosa em cenários onde as classes são desbalanceadas. Por isso, é crucial analisar a acurácia em conjunto com outras métricas, especialmente em problemas de saúde onde a detecção de casos positivos é de extrema importância ([HASTIE; TIBSHIRANI; FRIEDMAN, 2009](#)).

4.2.3 Loss

A função de perda, ou loss function, é um componente fundamental no treinamento de modelos de Machine Learning. Ela quantifica a diferença entre as previsões do modelo e os valores reais dos dados. O objetivo do treinamento de um modelo é minimizar essa função de perda, ajustando os pesos e vieses da rede para que suas previsões se tornem cada vez mais próximas dos valores reais (GOODFELLOW; BENGIO; COURVILLE, 2016).

A escolha da função de perda depende do tipo de problema que está sendo resolvido:

Para tarefas de classificação, funções de perda comuns incluem:

- **Binary Cross-Entropy (Entropia Cruzada Binária):** Utilizada para problemas de classification binária, onde há apenas duas classes possíveis.
- **Categorical Cross-Entropy (Entropia Cruzada Categórica):** Usada para problemas de classificação multiclasse, onde há mais de duas classes.

Para tarefas de regressão, funções de perda comuns incluem:

- **Mean Squared Error (MSE - Erro Quadrático Médio):** Calcula a média dos quadrados das diferenças entre os valores previstos e os valores reais.
- **Mean Absolute Error (MAE - Erro Absoluto Médio):** Calcula a média dos valores absolutos das diferenças entre os valores previstos e os valores reais.

Durante o treinamento, o algoritmo de otimização utiliza o valor da função de perda para determinar a direção e a magnitude dos ajustes nos parâmetros do modelo. Uma função de perda bem escolhida é crucial para guiar o modelo a aprender padrões significativos nos dados e a generalizar bem para dados não vistos.

4.2.4 Precisão

A Precisão é uma métrica que avalia a qualidade das previsões positivas do modelo. Uma alta precisão indica que o modelo tem poucos Falsos Positivos, ou seja, ele raramente classifica incorretamente uma instância negativa como positiva (FREITAS, 2024). A fórmula para calcular a Precisão é:

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{VP}{VP + FP}$$

Onde:

- VP = Verdadeiro Positivo
- FP = Falso Positivo

Em um contexto de predição de diabetes, uma alta precisão é importante para minimizar o número de falsos alarmes, evitando que indivíduos saudáveis sejam erroneamente diagnosticados com a doença, o que poderia gerar ansiedade desnecessária e custos com exames adicionais. No entanto, um modelo com alta precisão pode ter um baixo Recall, ou seja, pode deixar de identificar muitos casos reais de diabetes.

4.2.5 Recall

O Recall é uma métrica que avalia a capacidade do modelo de identificar todas as instâncias positivas reais. Ela responde à pergunta: “Das instâncias que realmente são positivas, quantas o modelo conseguiu identificar corretamente?”. Um alto Recall indica que o modelo tem poucos Falsos Negativos, ou seja, ele raramente deixa de identificar uma instância positiva (FREITAS, 2024). A fórmula para calcular o Recall é:

$$\text{Recall} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{VP}{VP + FN}$$

Onde:

- VP = Verdadeiro Positivo
- FN = Falso Negativo

Em um contexto de predição de diabetes, um alto Recall é crucial para garantir que a maioria dos indivíduos com a doença seja identificada, minimizando o risco de falsos negativos. Deixar de identificar um caso real de diabetes pode ter consequências graves para a saúde do paciente, pois o tratamento precoce é essencial. No entanto, um modelo com alto Recall pode ter uma baixa precisão, ou seja, pode gerar muitos falsos positivos.

4.2.6 F1 Score

O F1 Score é uma métrica que combina a Precisão e o Recall em uma única pontuação, sendo a média harmônica dessas duas métricas. Ele é particularmente útil quando há um desequilíbrio entre as classes no conjunto de dados (ou seja, uma classe é muito mais frequente que a outra) e quando Falsos Positivos e Falsos Negativos têm custos semelhantes. O F1 Score busca um equilíbrio entre Precisão e Recall, penalizando modelos que performam bem em uma métrica, mas mal na outra.

(??)Han2012). A fórmula para calcular o F1 Score é:

$$\text{F1 Score} = \frac{VP}{VP + \frac{VP+FN}{2}}$$

Onde:

- VP = Verdadeiro Positivo
- FN = Falso Negativo

Um F1 Score alto indica que o modelo tem tanto alta precisão quanto alto recall. Em problemas de saúde, onde tanto a identificação correta de pacientes doentes quanto a minimização de diagnósticos errados são importantes, o F1 Score pode ser uma métrica mais representativa do desempenho geral do modelo do que a acurácia isoladamente.

4.3 Construção Dos Modelos

A arquitetura central adotada neste estudo é a Multi-Layer Perceptron (MLP), cuja escolha é justificada por sua comprovada eficácia e ampla utilização como base-line robusto para problemas de classificação com dados tabulares (GOODFELLOW; BENGIO; COURVILLE, 2016). O MLP é intrinsecamente adequado para modelar relações não-lineares complexas, considerando o conjunto de features de forma global e totalmente conectadas.

Adicionalmente, com o objetivo estritamente comparativo, uma arquitetura de Rede Neural Convolutacional (CNN) foi implementada. Embora a CNN seja uma abordagem para domínios como visão computacional, sua aplicação em dados tabulares não é trivial, mas serve here como um contraponto metodológico. A inclusão da CNN visa contrastar o desempenho do modelo padrão com uma arquitetura de paradigma distinto, que opera através da extração de padrões locais. Esta comparação permite avaliar se o modelo MLP padrão é de fato suficiente para o problema ou se uma topologia não convencional, mesmo que não seja teoricamente a primeira escolha, pode oferecer resultados competitivos. A busca por hiperparâmetros foi empregada para otimizar e avaliar imparcialmente ambas as topologias, visando a seleção de um modelo eficaz.

4.3.1 Otimização Bayesiana Aplicada Seleção de Hiperparâmetros

Para a definição dos modelos de redes neurais, optou-se por um processo automatizado de ajuste de hiperparâmetros por meio da otimização bayesiana. Este método foi escolhido por sua eficiência superior quando comparado a abordagens como a busca aleatória.

A seleção de hiperparâmetros adequados representa um dos maiores desafios no desenvolvimento de redes neurais, uma vez que o espaço de busca cresce exponencialmente com o número de parâmetros configuráveis. Para este trabalho, implementou-se uma estratégia de otimização bayesiana que explora inteligentemente um vasto espaço de configurações possíveis para as redes neurais aplicadas à predição de diabetes.

A otimização bayesiana resolve esse problema através de uma abordagem probabilística que modela a relação entre hiperparâmetros e desempenho do modelo usando processos gaussianos. Diferentemente de métodos tradicionais como busca aleatória ou busca em grade, que tratam cada experimento de forma independente, o otimizador bayesiano aprende com os resultados anteriores para decidir quais configurações testar a seguir. O algoritmo mantém uma distribuição de probabilidade sobre o espaço de hiperparâmetros e utiliza uma função de aquisição para balancear exploitation e exploration.

A implementação utiliza o Keras Tuner com parâmetros específicos ajustados para o problema: 5 pontos iniciais exploratórios são gerados aleatoriamente para estabelecer uma base de conhecimento inicial, seguidos por seleção bayesiana guiada para os trials subsequentes. Os parâmetros alpha e beta da função de aquisição foram configurados para favorecer uma exploração moderada do espaço, evitando convergência prematura para ótimos locais. Cada configuração testada é avaliada com múltiplas execuções para considerar a variabilidade estocástica do treinamento de redes neurais.

Durante o processo de busca, o algoritmo executa cada trial por um número predefinido de épocas, coletando métricas de desempenho incluindo precisão, acurácia, AUC-ROC, recall e perda tanto para os conjuntos de treino quanto de validação. Um mecanismo de early stopping monitora a métrica objetivo e interrompe o treinamento caso não haja melhoria por 20 épocas consecutivas, economizando tempo computacional sem comprometer a qualidade da avaliação. Esta estratégia permite que configurações promissoras treinem até convergência enquanto configurações ruins são descartadas rapidamente.

A cada trial concluído, o otimizador bayesiano atualiza seu modelo probabilístico do espaço de hiperparâmetros, refinando sua compreensão sobre quais regiões são mais promissoras. Esta abordagem adaptativa permite que o algoritmo concentre seus recursos computacionais nas áreas do espaço de busca com maior probabilidade de conter a configuração ótima. Na prática, isso significa que enquanto uma busca aleatória de 50 trials poderia desperdiçar recursos testando configurações claramente subótimas, a otimização bayesiana direciona progressivamente seus testes para configurações cada vez mais refinadas.

Ao final da otimização, o sistema gera automaticamente estatísticas comparativas, as cinco melhores configurações e salva o modelo de melhor desempenho com seus hiperparâmetros em JSON, garantindo reprodutibilidade. A análise temporal registra o tempo médio por trial e o tempo total de execução.

A otimização Bayesiana supera a busca aleatória, atingindo configurações de alta performance mais rapidamente e com menos experimentos. Sua eficiência é crucial para o ajuste de hiperparâmetros em redes neurais, especialmente com recursos computacionais e tempo limitados.

Os Modelos Escolhidos

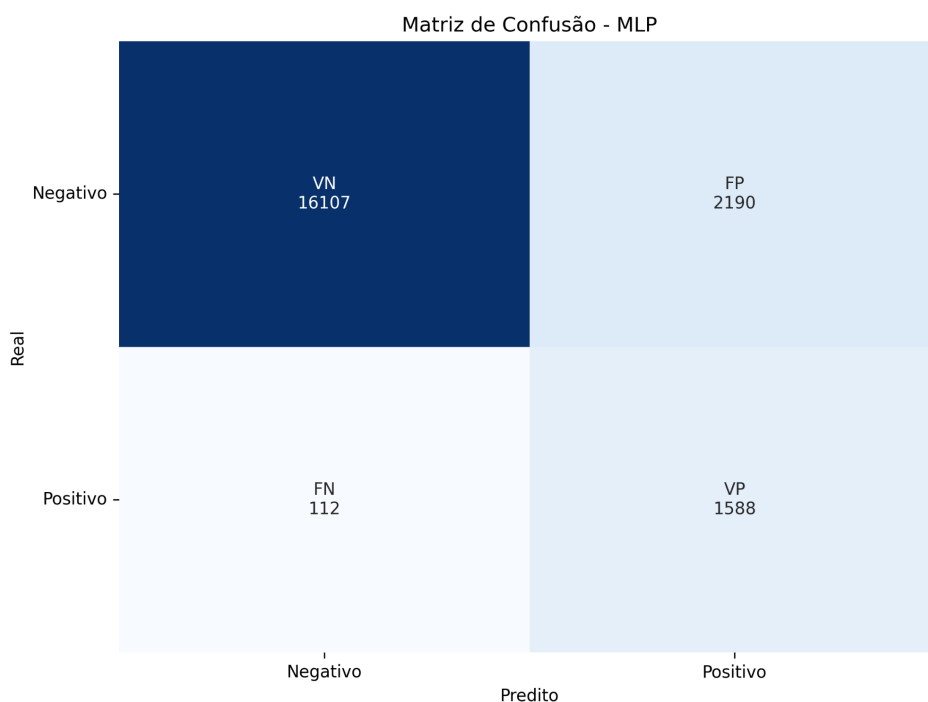
A otimização de hiperparâmetros revelou uma disparidade de desempenho substancial entre as arquiteturas propostas. O modelo MLP selecionado configurou-se como uma rede profunda de 5 camadas ocultas, caracterizada por uma arquitetura complexa, um misto de funções de ativação e uso intensivo de técnicas de regularização, como Batch Normalization em quatro camadas, altos níveis de Dropout e regularização L2. Utilizando o Nadam otimizador, este modelo atingiu 71.03% de precisão na validação e um recall de 80,82%.

Em contrapartida, o modelo CNN, incluído para fins comparativos, emergiu como a arquitetura superior, alcançando 86,93% de precisão na validação. A arquitetura vencedora é um modelo híbrido 1D-CNN composto por uma base convolucional de 3 camadas atuando como extratora de features, seguida por uma cabeça de classificação (MLP) densa de 2 camadas. Este modelo também utilizou forte regularização, mas aplicou Batch Normalization apenas nas camadas densas. Notavelmente, o melhor desempenho foi obtido com o otimizador SGD, uma escolha mais simples que o Nadam utilizado pelo MLP.

4.3.2 Avaliação dos modelos

Para uma análise de desempenho comparativa e aprofundada, indo além da métrica de acurácia, foram geradas Matrizes de Confusão para todos os modelos avaliados: MLP, CNN, RandomForestClassifier e GradientBoostingClassifier. Estas matrizes, apresentadas na figura 12, são cruciais no contexto do diagnóstico de diabetes, pois permitem avaliar os tipos de erros específicos que cada arquitetura comete.

Figura 12 – Matriz de Confusão.



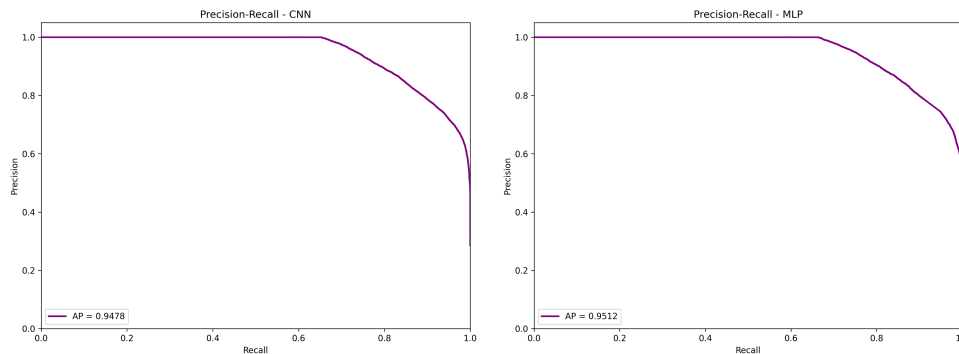
Fonte: Elaborado pelo autor

Na predição desta doença, existem dois erros principais a considerar: o Falso Positivo, que ocorre quando o modelo classifica incorretamente um paciente saudável como diabético, e o Falso Negativo, que ocorre quando o modelo falha em identificar um paciente que realmente possui a doença. Embora Falsos Positivos sejam indesejáveis por levarem a exames desnecessários, o Falso Negativo representa o cenário de maior risco clínico, pois um paciente doente é liberado sem diagnóstico, atrasando o tratamento e aumentando o risco de complicações. Portanto, a comparação do número de Falsos Negativos gerados por cada modelo será o fator determinante para avaliar qual deles oferece o maior potencial como ferramenta de apoio à decisão médica.

Métricas Para otimizar a seleção do modelo preditivo de diabetes, priorizamos métricas que visam reduzir os falsos negativos. Essa abordagem minimiza a chance de pacientes serem erroneamente descartados devido a previsões incorretas do modelo.

Os modelos foram treinados com o objetivo de otimizar a métrica de recall, buscando reduzir a quantidade de falsos positivos, sem comprometer a precisão das escolhas.

Figura 13 – Resultados: Curva Precision Recall.



Fonte: Elaborado pelo autor

Na figura 13 podemos ver um exemplo de uma curva precisão-recall ela ilustra a relação inversa entre precisão e recall em diferentes pontos de corte. Um alto valor de área sob a curva indica tanto um recall elevado quanto uma precisão alta. A alta precisão reflete a minimização de falsos positivos nos resultados apresentados, enquanto o alto recall garante que poucos falsos negativos sejam incluídos nos resultados relevantes. Consequentemente, pontuações elevadas em ambos os aspectos sugerem que o classificador oferece resultados exatos e abrange a maioria dos resultados pertinentes. (scikit-learn, 2025)

Esta métrica foi crucial na seleção do modelo final, visto que a minimização de Falsos Negativos era de suma importância para o problema em questão.

4.4 Resultados preliminares e Discussão

Discutiremos, na sequência, os resultados das explicações geradas pelos métodos aplicados no capítulo anterior.

4.4.1 Desempenho

Trials: Escolhendo o melhor modelo Aqui vamos mostrar os resultados obtidos. As Tabelas 2 e 3 exibem, respectivamente, o desempenho dos ensaios para os modelos MLP e CNN. No caso do modelo MLP, optou-se pelo ensaio 19, que apresentou um recall de 80,82% e precisão de 71,03%. Para o modelo CNN, o ensaio 19 também foi selecionado devido à sua precisão de 86,93%, que, apesar de um recall ligeiramente inferior ao do MLP, alinha-se com a estratégia de priorizar o recall.

Tabela 2 – Resultados Trials MPL

trial_id	precision	accuracy	recall	loss	auc
19	71.03%	95.53%	80.82%	12.48%	88.23%
7	60.62%	91.69%	84.71%	17.64%	86.91%
11	61.55%	94.19%	84.71%	13.02%	88.23%
14	64.57%	94.20%	83.76%	14.36%	88.07%
17	61.97%	93.93%	83.29%	14.48%	87.83%
10	69.99%	95.43%	80.94%	11.94%	88.04%
3	70.87%	95.54%	80.71%	13.04%	88.20%
12	64.00%	94.49%	80.47%	13.30%	85.85%
4	60.82%	93.90%	79.53%	21.38%	81.85%
16	75.75%	95.96%	78.71%	12.15%	88.11%
6	75.17%	95.96%	78.35%	14.53%	88.00%
15	76.53%	95.11%	78.12%	13.91%	87.93%
0	70.89%	95.36%	77.29%	13.64%	86.79%
2	74.95%	95.83%	77.18%	11.28%	87.82%
18	80.67%	96.25%	75.65%	13.35%	87.30%
8	79.16%	95.98%	74.24%	15.24%	86.73%
13	91.82%	97.10%	72.35%	14.67%	88.01%
5	93.95%	96.97%	69.29%	14.04%	87.36%
9	99.08%	96.73%	62.12%	25.46%	85.78%
1	69.14%	95.13%	77.18%	26.49%	85.68%

Tabela 3 – Resultados Trials CNN

trial_id	precision	accuracy	auc	recall	loss
19	86.93%	96.83%	97.48%	74.24%	13.28%
8	56.30%	93.08%	96.63%	83.18%	16.45%
9	63.16%	94.41%	97.54%	82.35%	23.15%
14	66.52%	94.91%	97.50%	80.82%	14.10%
1	65.17%	94.58%	97.12%	79.65%	51.37%
18	68.08%	95.01%	97.38%	78.00%	13.56%
15	66.77%	94.82%	97.00%	77.76%	27.38%
10	70.53%	95.25%	97.06%	76.12%	28.55%
2	75.47%	95.77%	97.30%	74.47%	13.59%
16	71.46%	95.28%	96.84%	74.12%	22.60%
3	89.50%	96.92%	97.64%	72.35%	12.86%
0	83.10%	96.35%	97.40%	71.65%	13.26%
5	85.17%	96.53%	97.54%	71.65%	12.84%
6	90.11%	96.76%	97.14%	70.12%	13.75%
17	88.71%	96.66%	97.33%	69.65%	12.63%
4	80.34%	95.86%	96.97%	69.18%	15.24%
12	79.76%	95.81%	96.60%	68.00%	28.63%
7	89.57%	96.40%	97.16%	65.18%	15.55%
13	87.31%	95.64%	96.23%	57.53%	15.57%

Consequentemente, os modelos escolhidos para as análises comparativas futuras, que foram apresentados na seção 4.2.1.1, foram cuidadosamente selecionados com base em critérios rigorosos, incluindo a métrica de recall previamente discutida em parágrafos anteriores. Essa seleção visa garantir a relevância e a robustez dos resultados obtidos. Desse modo, a metodologia adotada para a escolha desses modelos é um pilar fundamental dessa pesquisa.

4.4.2 Treinamento

Após a fase de ajuste, que determinou as configurações ideais de hiperparâmetros, o retreinamento foi realizado no conjunto de treinamento com validação retida. Durante este processo, aplicou-se regularização e controle de sobreajuste por meio de EarlyStopping, que monitorou a `pr_auc`, e Model Checkpoint, que salvou o modelo com melhor desempenho com base no mesmo critério. Adicionalmente, utilizou-se ReduceLROnPlateau para o ajuste adaptativo da taxa de aprendizado. Quando necessário, foram incorporados pesos de classe para corrigir assimetrias nas frequências das classes. Ao final, os modelos "best" e "final" foram salvos para garantir a reprodutibilidade. A avaliação de desempenho foi conduzida em um conjunto de teste separado, reportando métricas padronizadas como precisão, revocação, F1 e AUC-ROC. Este protocolo assegura a reprodutibilidade, rastreabilidade e validade externa, em conformidade com as boas práticas de engenharia e experimentação em aprendizado de máquina.

Os resultados dos treinamentos, para CNN e MLP respectivamente, estão apresentados nas tabelas 4 e 5.

Tabela 4 – Resultados do treinamento CNN

metric	mean	std	min	max
loss	23.91%	0.61%	23.48%	28.48%
accuracy	88.96%	0.17%	87.83%	89.16%
precision	75.41%	0.31%	73.49%	75.80%
recall	91.05%	0.21%	89.79%	91.37%
auc	97.04%	0.11%	96.19%	97.11%

Tabela 5 – Resultados do treinamento MLP

metric	mean	std	min	max
loss	21.64%	1.85%	19.60%	29.04%
accuracy	89.11%	0.57%	87.56%	89.88%
precision	75.37%	1.06%	73.00%	76.84%
recall	91.98%	0.47%	89.62%	92.65%
auc	97.18%	0.32%	96.06%	97.55%

Observa-se que o modelo MLP superou ligeiramente o modelo que utiliza a arquitetura CNN, embora a diferença não seja muito significativa.

5 Conclusão

Este trabalho de conclusão de curso abordou o desafio da predição de diabetes tipo 2, um problema de saúde pública de crescente relevância e prevalência global. O objetivo central foi desenvolver e avaliar modelos de redes neurais artificiais capazes de funcionar como uma ferramenta de apoio ao diagnóstico precoce, utilizando dados clínicos e comportamentais acessíveis.

O problema foi tratado através da construção de um pipeline de *Machine Learning* robusto, aplicado a um *dataset* público de 100.000 registros. Um desafio metodológico central foi o significativo desbalanceamento de classes do *dataset* (92

Foram implementadas e comparadas duas arquiteturas de redes neurais, uma *Multi-Layer Perceptron* (MLP) e uma Rede Neural Convolucional (CNN) 1D , além de modelos clássicos de *Machine Learning* (Random Forest e Gradient Boosting) para estabelecer um *baseline*. A otimização bayesiana foi utilizada para o ajuste fino dos hiperparâmetros das redes neurais, garantindo uma exploração eficiente do espaço de busca. Os resultados obtidos foram promissores: os modelos de redes neurais alcançaram acurácia elevada, com o MLP atingindo 89,11

A contribuição deste trabalho pessoalmente, a execução deste projeto exigiu uma imersão profunda nas complexidades da aplicação de inteligência artificial em um contexto de alto impacto social. O desafio de tratar dados desbalanceados , implementar e otimizar arquiteturas de *Deep Learning* , e interpretar os resultados sob a ótica da responsabilidade ética (priorizando o *recall* para minimizar o risco clínico), solidificou competências técnicas em ciência de dados, engenharia de *Machine Learning* e pensamento crítico.

Finalmente, este trabalho cumpre seus objetivos ao entregar modelos funcionais e ao validar a hipótese de que redes neurais são ferramentas eficazes para esta tarefa. Conforme detalhado na seção de Trabalhos Futuros , o pipeline desenvolvido serve como alicerce para a criação de uma plataforma interativa e a validação com *datasets* mais abrangentes, reforçando o potencial de aplicação prática da solução.

Referências

ABRAHAM, A. Artificial neural networks. In: SYDENHAM, P. H.; THORN, R. (Ed.). *Handbook of Measuring System Design*. London: John Wiley & Sons, 2005. p. 901–908. Citado 2 vezes nas páginas 25 e 26.

ALZUBAIDI, L. et al. Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, v. 8, n. 1, p. 1–74, 2021. Citado na página 23.

ARAÚJO, L. F. d. *Uma abordagem baseada em redes neurais artificiais para diagnóstico de diabetes*. Macapá, 2021. Citado na página 39.

BASSO, M. et al. Sistema inteligente para apoio ao diagnóstico de diabetes empregando redes neurais. In: *Anais do EATI - Encontro Anual de Tecnologia da Informação*. [S.l.: s.n.], 2014. p. 56–63. In: ENCONTRO ANUAL DE TECNOLOGIA DA INFORMAÇÃO, [2014], UFSM. Citado na página 40.

BISHOP, C. M. *Pattern Recognition and Machine Learning*. [S.l.]: Springer, 2006. Citado na página 46.

BRASIL. *Diabetes (Diabetes Mellitus)*. 2022. Ministério da Saúde. Disponível em: <<https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/d/diabetes>>. Acesso em: [Data de Acesso]. Usado para a classificação dos tipos de diabetes. Citado 2 vezes nas páginas 21 e 38.

BURKOV, A. *Machine Learning Engineering*. [S.l.]: True Positive Inc., 2020. Citado na página 21.

BUSNARDO, N. G. Detecção de fake news utilizando redes neurais convolucionais. Rio do Sul, 2024. Citado 3 vezes nas páginas 24, 25 e 26.

CHAPELLE, O.; SCHOLKOPF, B.; ZIEN, A. *Semi-Supervised Learning*. [S.l.]: MIT Press, 2006. Citado na página 22.

DEO, R. C. Machine learning in medicine. *Circulation*, v. 132, n. 20, p. 1920–1930, 2015. Citado na página 17.

FEITOSA, R. D. F.; SOUZA, D. T. d.; SOUZA, R. T. *Introdução a Machine Learning e redes neurais*. [S.l.]: Cegraf UFG, 2024. Citado na página 22.

FREITAS, N. C. A. d. *Machine learning: técnicas e cases*. 1. ed. [S.l.]: Novatec, 2024. Citado 3 vezes nas páginas 46, 48 e 49.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. Citado 3 vezes nas páginas 27, 48 e 50.

HAN, J.; KAMBER, M.; PEI, J. *Data Mining: Concepts and Techniques*. 3. ed. [S.l.]: Morgan Kaufmann, 2012. Citado na página 46.

- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. ed. [S.l.]: Springer, 2009. Citado 2 vezes nas páginas 22 e 47.
- HAYKIN, S. *Neural Networks and Learning Machines*. 3. ed. Upper Saddle River, NJ: Pearson Education, 2009. Citado 2 vezes nas páginas 24 e 25.
- IAMMUSTAFA. *Diabetes prediction dataset*. 2022. Kaggle. Disponível em: <<https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset/data>>. Citado na página 43.
- IDF. *IDF Diabetes Atlas*. 10. ed. [S.l.]: IDF, 2021. Disponível em: <https://profissional.diabetes.org.br/wp-content/uploads/2022/02/IDF_Atlas_10th_Edition_2021-.pdf>. Citado 2 vezes nas páginas 17 e 18.
- ISLAM, S.; ISLAM, M. M. Diabetes prediction: A deep learning approach. *International Journal of Information Engineering and Electronic Business*, v. 11, n. 2, p. 21–27, 2019. Citado na página 40.
- KAVAKIOTIS, I. et al. Machine learning and data mining methods in diabetes research. *Computational and Structural Biotechnology Journal*, v. 15, p. 104–116, 2017. Citado 2 vezes nas páginas 17 e 18.
- KOVALESKI, P. d. A. et al. *Implementação De Redes Neurais Profundas Para Reconhecimento De Ações Vídeos*. [S.l.], 2018. Citado na página 26.
- LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, p. 436–444, 2015. Citado na página 26.
- MCKINNEY, W. Data structures for statistical computing in Python. In: *Proceedings of the 9th Python in Science Conference*. [S.l.: s.n.], 2010. v. 445, p. 56–61. Citado na página 32.
- MIOTTO, R. et al. Deep learning for healthcare: review, opportunities and challenges. *Briefings in Bioinformatics*, v. 19, n. 6, p. 1236–1246, 2018. Citado na página 17.
- MITCHELL, T. M. *Machine Learning*. [S.l.]: McGraw Hill, 1997. Citado na página 23.
- PEDREGOSA, F. et al. Scikit-learn: machine learning in python. *J. Mach. Learn. Res.*, v. 12, p. 2825, 2011. Citado na página 36.
- PIGHINI, H. D. *Um estudo sobre métricas para a avaliação de atributos*. [S.l.], 2025. Citado na página 47.
- PRADO, H. A. d. *Orpheo: Uma Estrutura de Trabalho para Integração dos Paradigmas de Aprendizado Supervisionado e Não-Supervisionado*. Tese (Doutorado) — PPGC da UFRGS, 2001. Citado na página 21.
- PYLINTON, L. Análise de IDEs para desenvolvimento em Python: PyCharm, VSCode e Spyder. *Revista Brasileira de Computação Aplicada*, v. 13, n. 2, p. 23–35, 2021. Citado na página 29.
- ROCHA, D. A. S. *Python para desenvolvedores*. 3. ed. [S.l.]: Novatec, 2021. Citado na página 29.

SANTOS, A. M. Usando redes neurais artificiais e regressão logística na predição da hepatite a. *Revista Brasileira de Epidemiologia*, v. 8, n. 2, p. 117–126, 2005. Citado na página 24.

SANTOS, D. P. d. *Tecnologia aplicada à saúde: uso de inteligência artificial na predição de diabetes para adultos*. [S.l.], 2024. Citado na página 39.

SBD. *Dados Epidemiológicos do diabetes mellitus no Brasil*. 2025. Disponível em: <https://diabetes.org.br/wp-content/uploads/2025/02/Dados-Epidemiologicos-SBD_comT1Dindex_2024_pdf.pdf>. Citado na página 17.

SISODIA, D.; SISODIA, D. S. Prediction of diabetes using classification algorithms. *Procedia Computer Science*, v. 132, p. 1578–1585, 2018. Citado 2 vezes nas páginas 17 e 18.

SPÖRL, C.; CASTRO, E.; LUCHIARI, A. Aplicação de redes neurais artificiais na construção de modelos de fragilidade ambiental. *Geography Department, University of Sao Paulo*, p. 113–135, 01 2011. Citado 2 vezes nas páginas 24 e 25.

STRAUSS, E.; JÚNIOR, M. V. B.; FERREIRA, W. L. L. A importância de utilizar métricas adequadas de avaliação de performance em modelos preditivos de machine learning. *Projectus*, v. 7, n. 4, p. 1–13, 2022. Citado na página 46.

TRASK, A. W. *Grokking Deep Learning*. [S.l.]: Manning Publications, 2019. Citado na página 26.

WHO. *Global report on diabetes*. [S.l.]: WHO, 2016. Disponível em: <<https://www.who.int/publications/i/item/9789241565257>>. Citado na página 17.

WORLD HEALTH ORGANIZATION (WHO). *Diabetes*. 2021. Genebra: WHO. Disponível em: <<https://www.who.int/news-room/fact-sheets/detail/diabetes>>. Citado na página 21.

APÊNDICE A – trials CNN

Tabela 6 – Resultados do tuning do modelo CNN

ID	Trial	Época	Precisão	Acurácia	AUC	Recall	Perda (Loss)
00	0	7	0.8310	0.9635	0.9740	0.7165	0.1326
01	1	9	0.6517	0.9458	0.9712	0.7965	0.5137
02	2	10	0.7547	0.9577	0.9730	0.7447	0.1359
03	3	10	0.8950	0.9692	0.9764	0.7235	0.1286
04	4	21	0.8034	0.9586	0.9697	0.6918	0.1524
05	5	12	0.8517	0.9653	0.9754	0.7165	0.1284
06	6	2	0.9011	0.9676	0.9714	0.7012	0.1375
07	7	11	0.8957	0.9640	0.9716	0.6518	0.1555
08	8	23	0.5630	0.9308	0.9663	0.8318	0.1645
09	9	22	0.6316	0.9441	0.9754	0.8235	0.2315
10	10	5	0.7053	0.9525	0.9706	0.7612	0.2855
11	11	0	N/A	N/A	N/A	N/A	N/A
12	12	23	0.7976	0.9581	0.9660	0.6800	0.2863
13	13	4	0.8731	0.9564	0.9623	0.5753	0.1557
14	14	22	0.6652	0.9491	0.9750	0.8082	0.1410
15	15	10	0.6677	0.9482	0.9700	0.7776	0.2738
16	16	9	0.7146	0.9528	0.9684	0.7412	0.2260
17	17	8	0.8871	0.9666	0.9733	0.6965	0.1263
18	18	16	0.6808	0.9501	0.9738	0.7800	0.1356
19	19	6	0.8693	0.9683	0.9748	0.7424	0.1328

APÊNDICE B – trials MLP

Tabela 7 – Resultados do tuning do modelo MLP

ID	Trial	Época	Precisão	PR AUC	Acurácia	AUC	Recall	Perda (Loss)
00	0	8	0.7089	0.8679	0.9536	0.9749	0.7729	0.1364
01	1	24	0.6914	0.8568	0.9513	0.9720	0.7718	0.2649
02	2	16	0.7495	0.8782	0.9583	0.9769	0.7718	0.1128
03	3	14	0.7087	0.8820	0.9554	0.9772	0.8071	0.1304
04	4	24	0.6082	0.8185	0.9390	0.9640	0.7953	0.2138
05	5	21	0.9395	0.8736	0.9697	0.9752	0.6929	0.1404
06	6	8	0.7517	0.8800	0.9596	0.9767	0.7835	0.1453
07	7	16	0.6062	0.8691	0.9169	0.9750	0.8471	0.1764
08	8	15	0.7916	0.8673	0.9598	0.9741	0.7424	0.1524
09	9	21	0.9908	0.8578	0.9673	0.9717	0.6212	0.2546
10	10	18	0.6999	0.8804	0.9543	0.9769	0.8094	0.1194
11	11	17	0.6155	0.8823	0.9419	0.9771	0.8471	0.1302
12	12	24	0.6400	0.8585	0.9449	0.9732	0.8047	0.1330
13	13	18	0.9182	0.8801	0.9710	0.9771	0.7235	0.1467
14	14	18	0.6457	0.8807	0.9420	0.9767	0.8376	0.1436
15	15	13	0.7653	0.8793	0.9511	0.9764	0.7812	0.1391
16	16	15	0.7575	0.8811	0.9596	0.9765	0.7871	0.1215
17	17	17	0.6197	0.8783	0.9393	0.9765	0.8329	0.1448
18	18	20	0.8067	0.8730	0.9625	0.9750	0.7565	0.1335
19	19	17	0.7103	0.8823	0.9553	0.9772	0.8082	0.1248